

IITMO

Введение в GameDev: ОСНОВЫ ИГРОВОГО ИИ

Лекция 3. Навигация и технологии программирования ИИИ

-  
1. Навигация
 2. Технологии программирования игрового ИИ
 - Восприятие
 - Данные и знания
 - Blackboard
 - Карты влияния
 - EQS
 3. Планирование: GOAP и HNT

ГЛАВА 3 _ Навигация

ІТМО



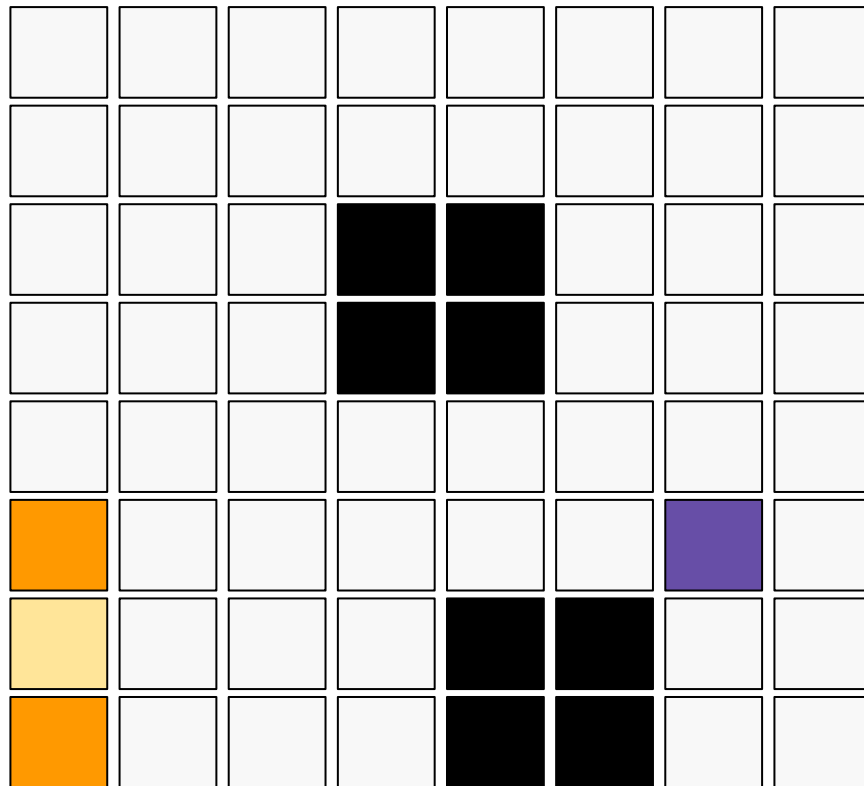
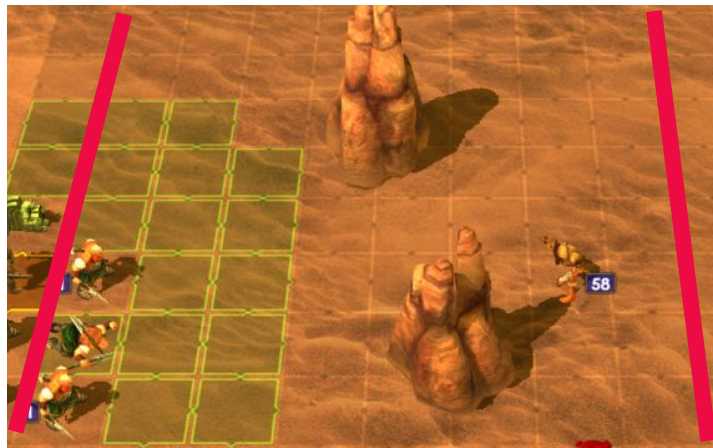
ГЛАВА 3 _ Навигация

ІІТМО



ГЛАВА 3 _ Навигация

ІТМО

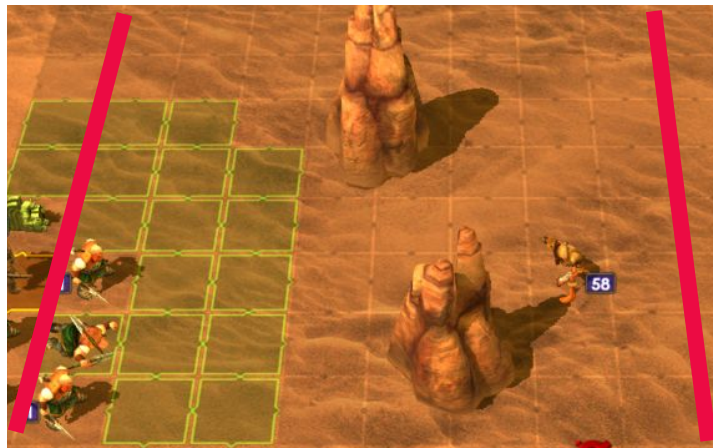


Goblin witch-doctor

Stats		
Attack	Defense	Hit Points
2	1	5
Damage	Initiative	Speed
1-2	9	5
Growth		
25		

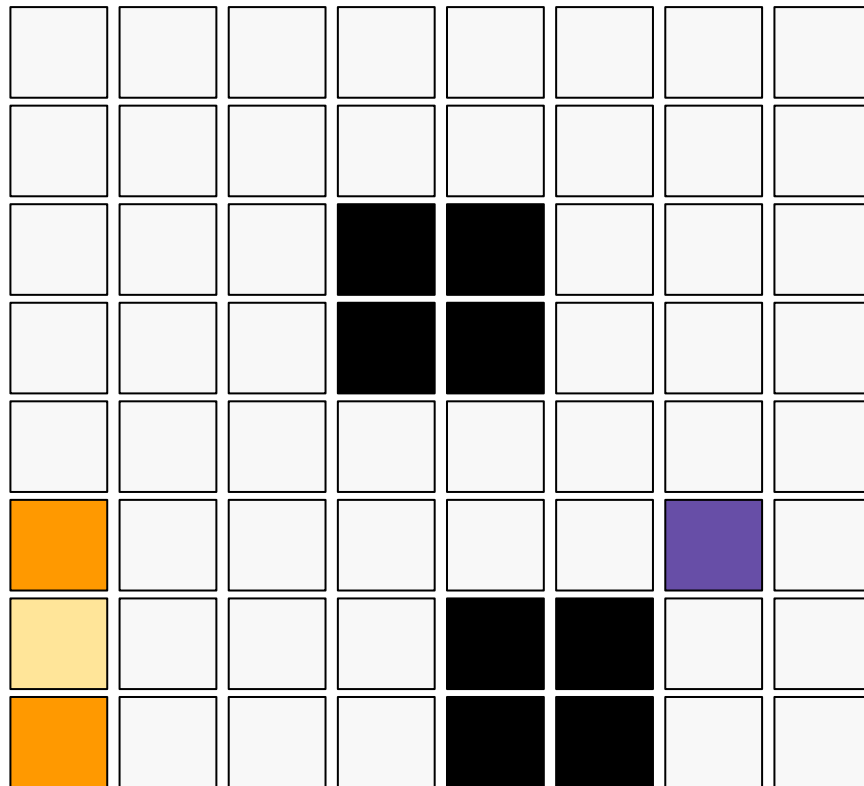
ГЛАВА 3 _ Навигация

ИТМО



Goblin witch-doctor

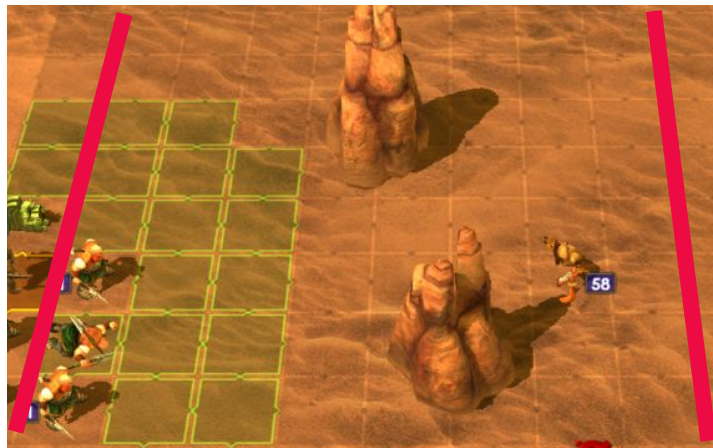
Stats		
Attack	Defense	Hit Points
2	1	5
Damage	Initiative	Speed
1-2	9	5
Growth		
25		



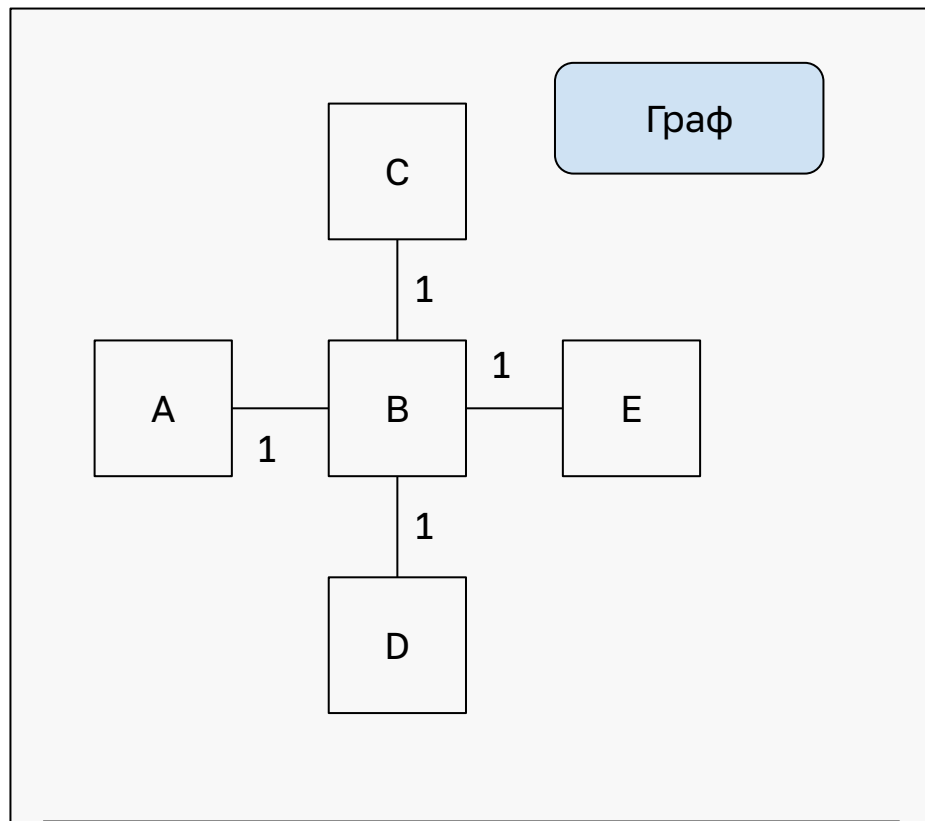
Как нарисовать возможные мувы шаману?

ГЛАВА 3 _ Навигация

ІТМО



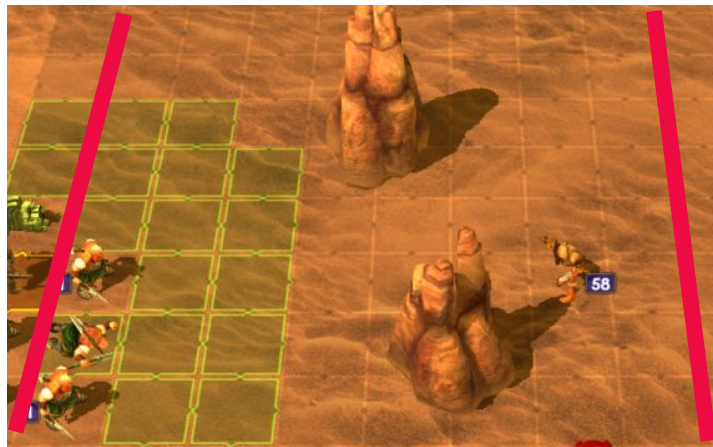
Stats ^		
Attack	Defense	Hit Points
2	1	5
Damage	Initiative	Speed
1-2	9	5
Growth		
25		



Как нарисовать возможные мувы шаману?

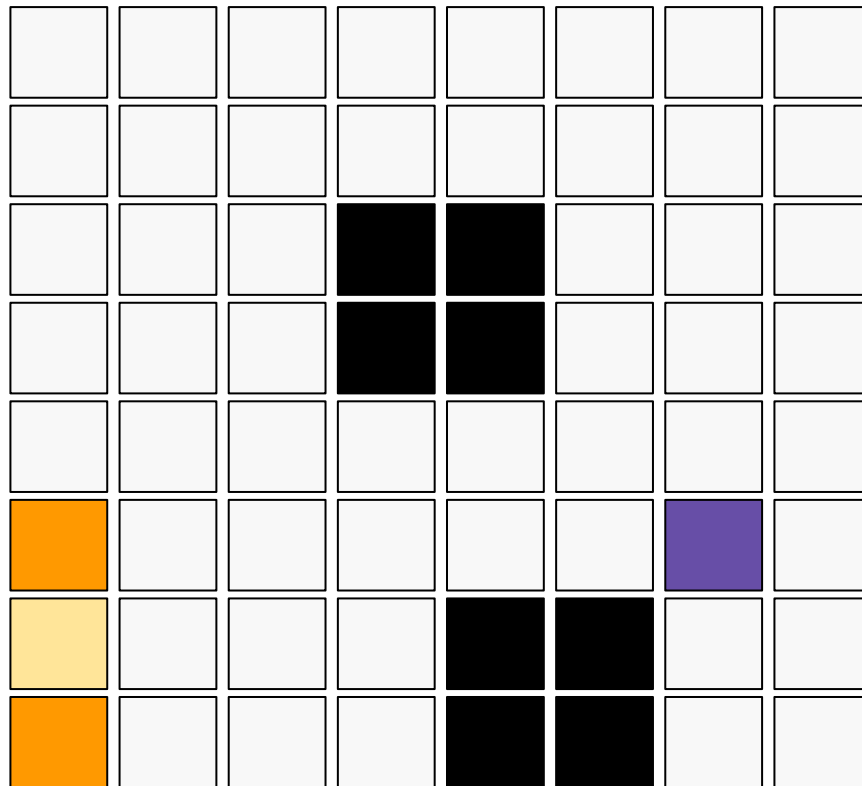
ГЛАВА 3 _ Навигация

ІТМО



Goblin witch-doctor

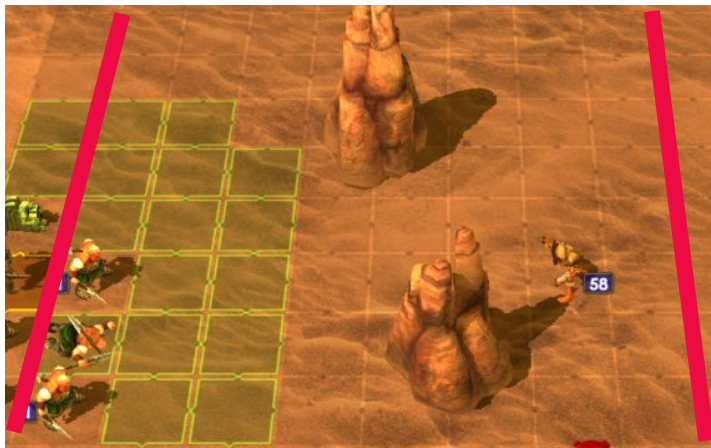
Stats		
Attack	Defense	Hit Points
2	1	5
Damage	Initiative	Speed
1-2	9	5
Growth		
25		



Поиск в ширину

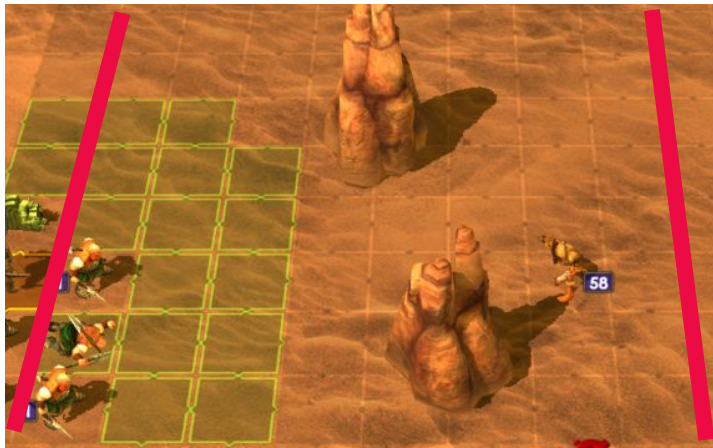
ГЛАВА 3 _ Навигация

ІТМО

[illegible]

Поиск в ширину

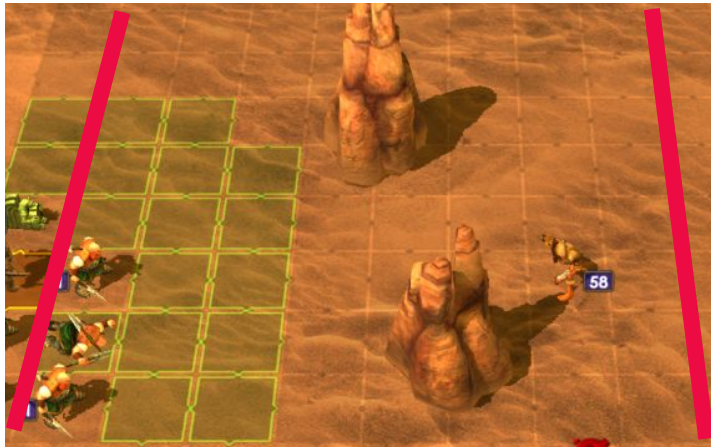
ГЛАВА 3 _ Навигация

ІТМО[illegible]

Поиск в ширину

ГЛАВА 3 _ Навигация

ІТМО

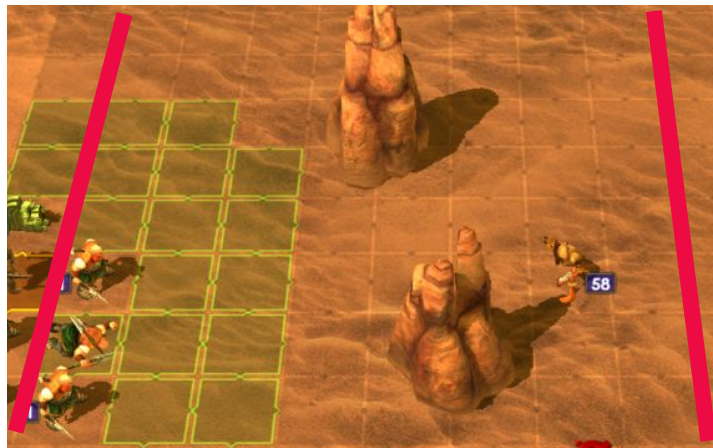


Stats		
Attack	Defense	Hit Points
2 	1 	5 
Damage	Initiative	Speed
1-2 	9 	5 
Growth		
25 		

[illegible]

Поиск в ширину

ГЛАВА 3 _ Навигация



Stats		
Attack	Defense	Hit Points
2	1	5
Damage	Initiative	Speed
1-2	9	5
Growth		
25		

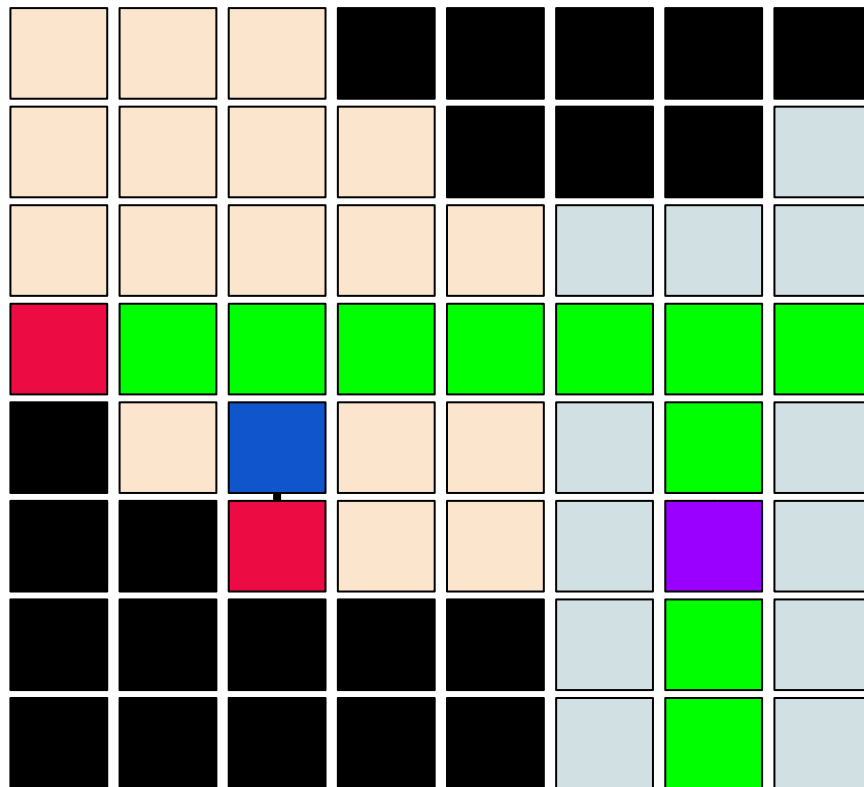
						5	
					5	4	5
					4	3	4
					3	2	3
		5	4	3	2	1	2
	5	4	3	2	1		1
		5	4			1	2
			5			2	3

Поиск в ширину

ГЛАВА 3 _ Навигация



Правила (стоимость):
Дорога – 1 ОД
“Своя” местность – 2 ОД
“Чужая” местность – 4 ОД



ГЛАВА 3 _ Навигация

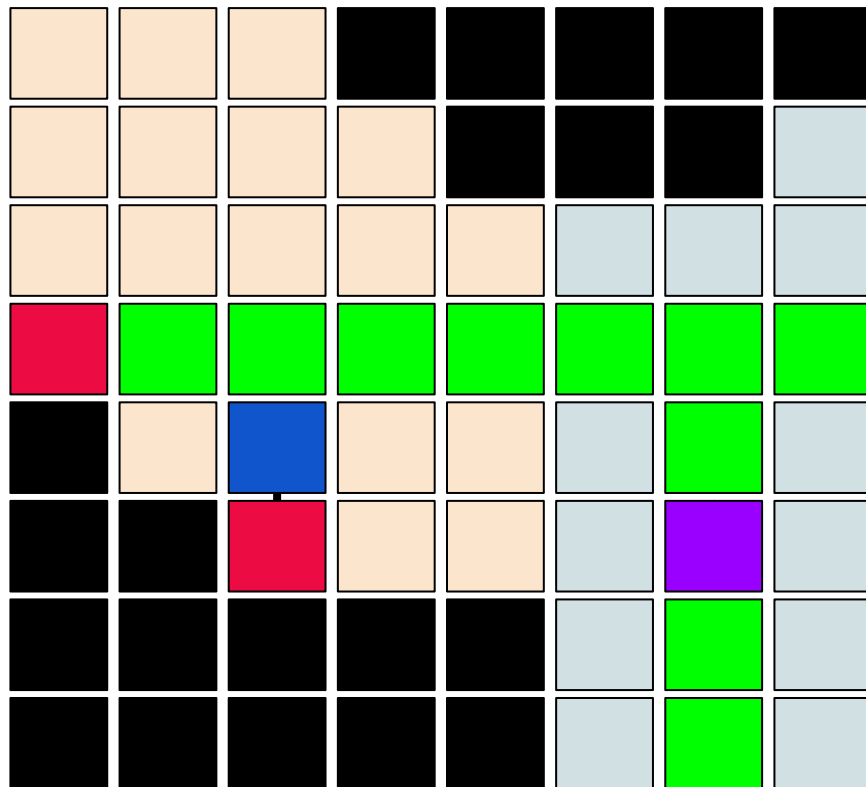


Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД



Наш герой хочет получить заклинание. Как проложить маршрут?

ГЛАВА 3 _ Навигация

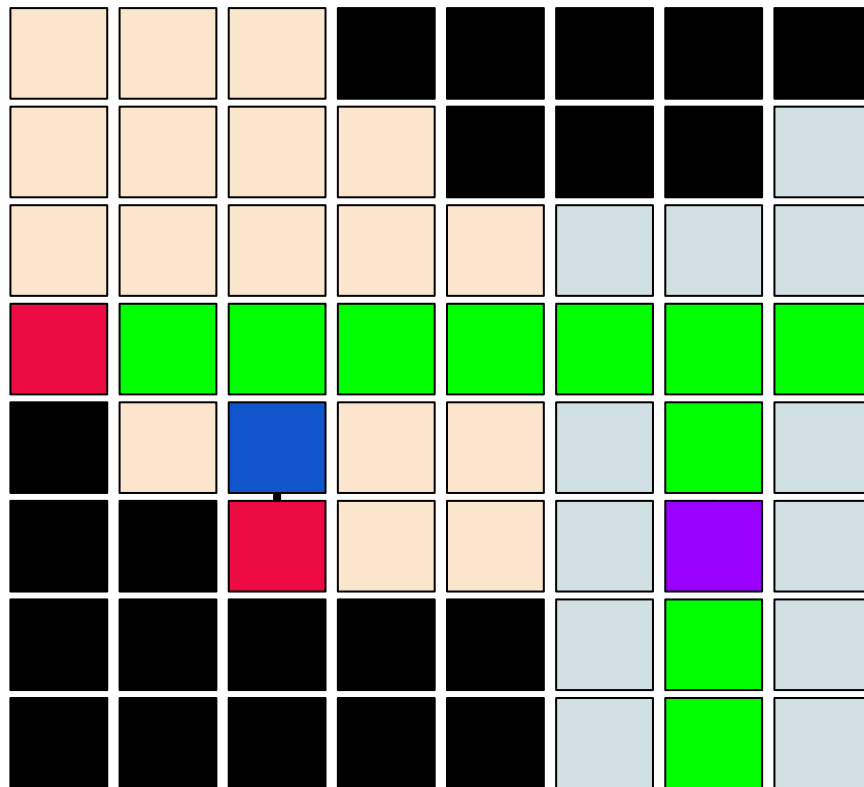


Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД



Как считаете, сработает поиск в ширину?

ГЛАВА 3 _ Навигация

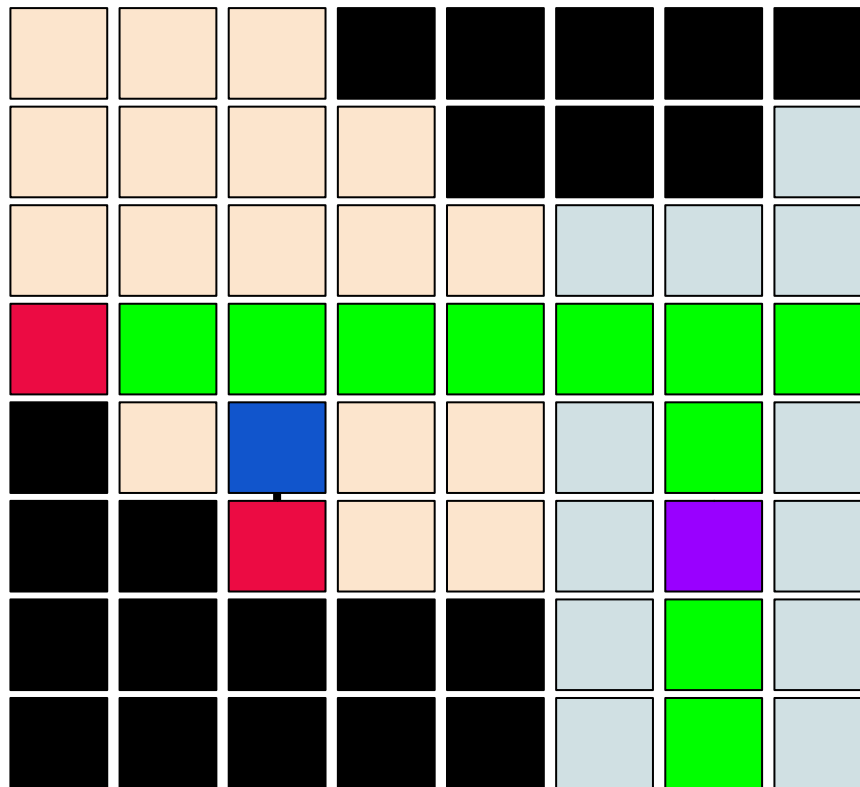


Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

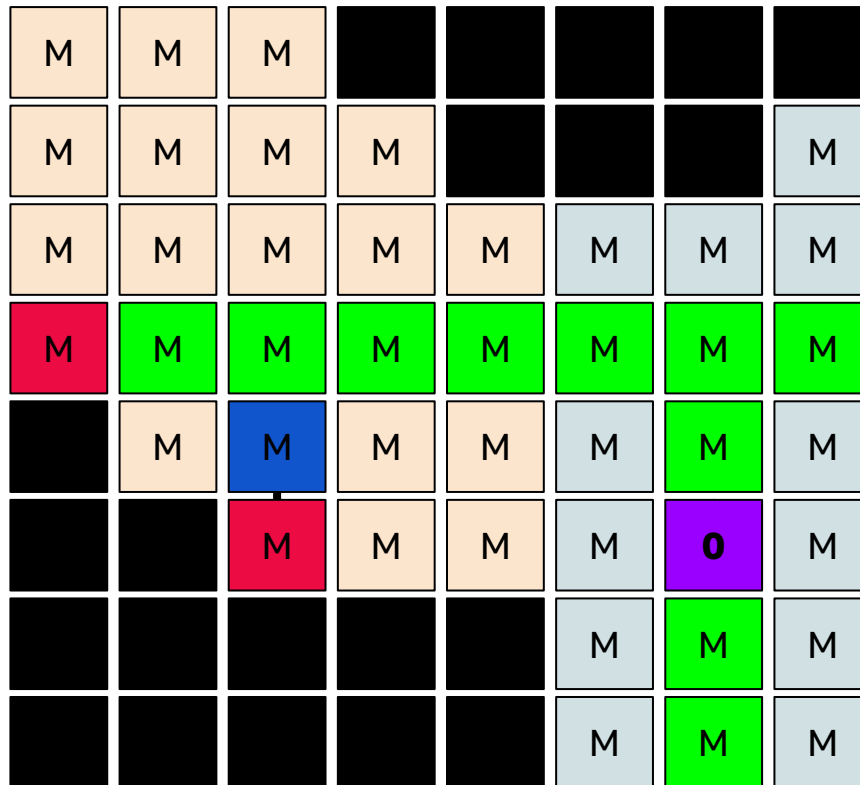


Новый алгоритм: Алгоритм Дейкстры для поиска кратчайшего пути!

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)

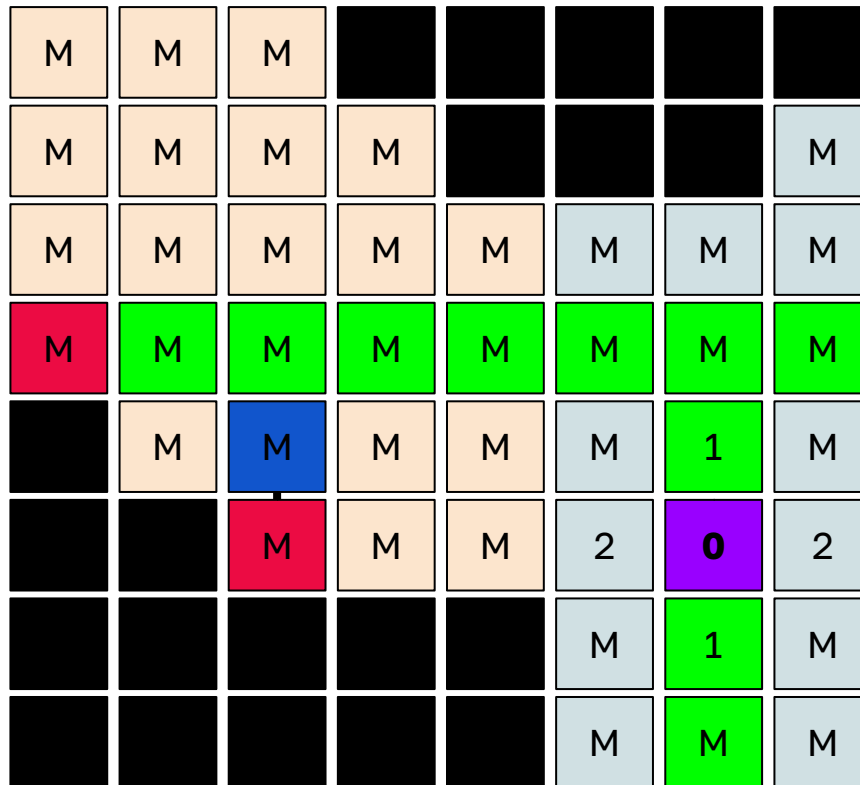
Новая ВМ = мин (прошлая ВМ, значение в рассматриваемой вершине + значение ребра)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД



ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

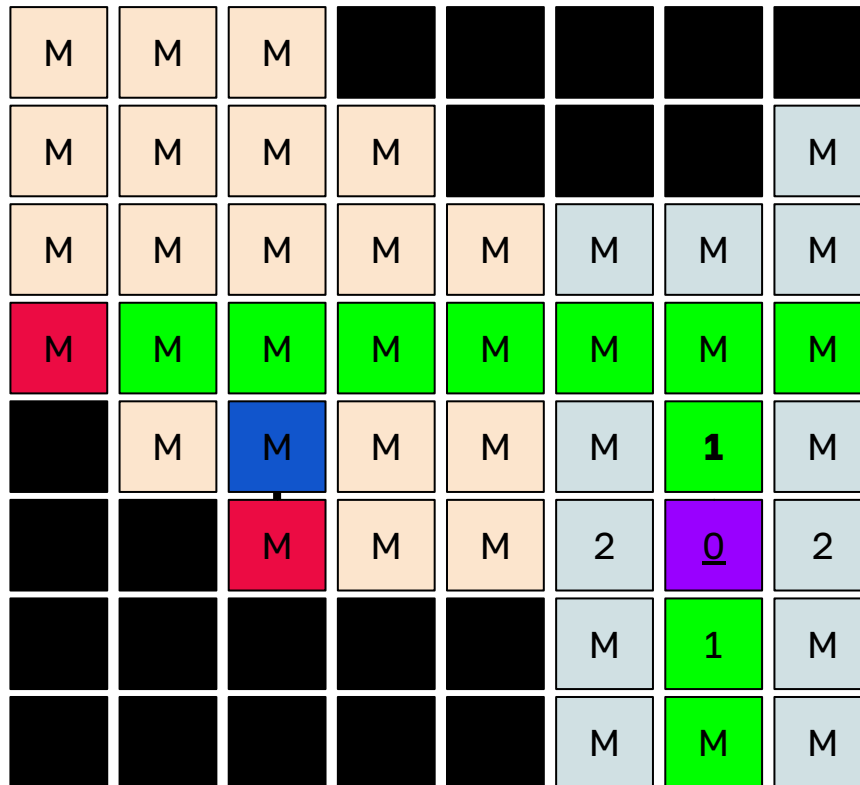
1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД



ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	М	М
М	М	М	М	М	М	2	М
	М	М	М	М	2	1	2
		М	М	М	2	0	2
					М	1	М
					М	М	М

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	М	М
М	М	М	М	М	М	2	М
	М	М	М	М	2	<u>1</u>	2
		М	М	М	2	<u>0</u>	2
					М	1	М
					М	М	М

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	М	М
М	М	М	М	М	М	2	М
	М	М	М	М	2	<u>1</u>	2
		М	М	М	2	<u>0</u>	2
					2	<u>1</u>	2
					М	2	М

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	М	М
М	М	М	М	М	3	2	М
	М	М	М	6	2	1	2
		М	М	М	2	0	2
					2	1	2
					М	2	М

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	4	М
М	М	М	М	М	3	2	3
	М	М	М	6	2	1	2
		М	М	М	2	0	2
					2	1	2
					М	2	М

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	М	4	М
М	М	М	М	М	3	2	3
	М	М	М	6	2	1	2
		М	М	6	2	0	2
					2	1	2
					4	2	4

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				М
М	М	М	М	М	5	<u>4</u>	5
М	М	М	М	4	<u>3</u>	<u>2</u>	<u>3</u>
	М	М	М	6	<u>2</u>	<u>1</u>	<u>2</u>
		М	М	6	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				7
М	М	М	М	8	<u>5</u>	<u>4</u>	<u>5</u>
М	М	М	5	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	М	М	М	6	<u>2</u>	<u>1</u>	<u>2</u>
		М	М	6	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				7
М	М	М	9	8	<u>5</u>	<u>4</u>	<u>5</u>
М	М	6	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	М	М	9	6	<u>2</u>	<u>1</u>	<u>2</u>
		М	М	6	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				7
М	М	М	9	8	<u>5</u>	<u>4</u>	<u>5</u>
М	М	6	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	М	М	9	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		М	10	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				7
М	М	10	9	8	5	4	5
М	7	6	5	4	3	2	3
	М	М	9	6	2	1	2
		М	10	6	2	0	2
					2	1	2
					4	2	4

Вот только сейчас метка стала постоянной!

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	М				<u>7</u>
М	11	10	9	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
17	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	11	М	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		М	10	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	М	13				<u>7</u>
М	11	10	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
17	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	11	М	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		М	10	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	14	13				<u>7</u>
М	11	<u>10</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
17	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	11	М	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		М	10	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

И так далее

Алгоритм Дейкстры

1. Временные метки М, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

М	М	М					
М	М	14	13				<u>7</u>
М	11	<u>10</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
17	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	11	М	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		20	<u>10</u>	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки M, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

24	<u>19</u>	18					
<u>19</u>	<u>15</u>	<u>14</u>	<u>13</u>				<u>7</u>
<u>15</u>	<u>11</u>	<u>10</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
<u>17</u>	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	<u>11</u>	<u>21</u>	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		<u>20</u>	<u>10</u>	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

Напасть на врага – 10 ОД; Посетить – 1 ОД.

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки M, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Вопрос: В чем сложность алгоритма Дейкстра?

<u>24</u>	<u>19</u>	18					
<u>19</u>	<u>15</u>	<u>14</u>	<u>13</u>				<u>7</u>
<u>15</u>	<u>11</u>	<u>10</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
<u>17</u>	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	<u>11</u>	<u>21</u>	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		<u>20</u>	<u>10</u>	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация

Алгоритм Дейкстры

1. Временные метки M, отсчет 0.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Алгоритм Дейкста ищет кратчайший путь от одной точки до всех остальных! То есть также рассматривает кратчайшие пути во все стороны!

<u>24</u>	<u>19</u>	18					
<u>19</u>	<u>15</u>	<u>14</u>	<u>13</u>				<u>7</u>
<u>15</u>	<u>11</u>	<u>10</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>4</u>	<u>5</u>
<u>17</u>	<u>7</u>	<u>6</u>	<u>5</u>	<u>4</u>	<u>3</u>	<u>2</u>	<u>3</u>
	<u>11</u>	<u>21</u>	<u>9</u>	<u>6</u>	<u>2</u>	<u>1</u>	<u>2</u>
		<u>20</u>	<u>10</u>	<u>6</u>	<u>2</u>	<u>0</u>	<u>2</u>
					<u>2</u>	<u>1</u>	<u>2</u>
					<u>4</u>	<u>2</u>	<u>4</u>

ГЛАВА 3 _ Навигация



Правила (стоимость):

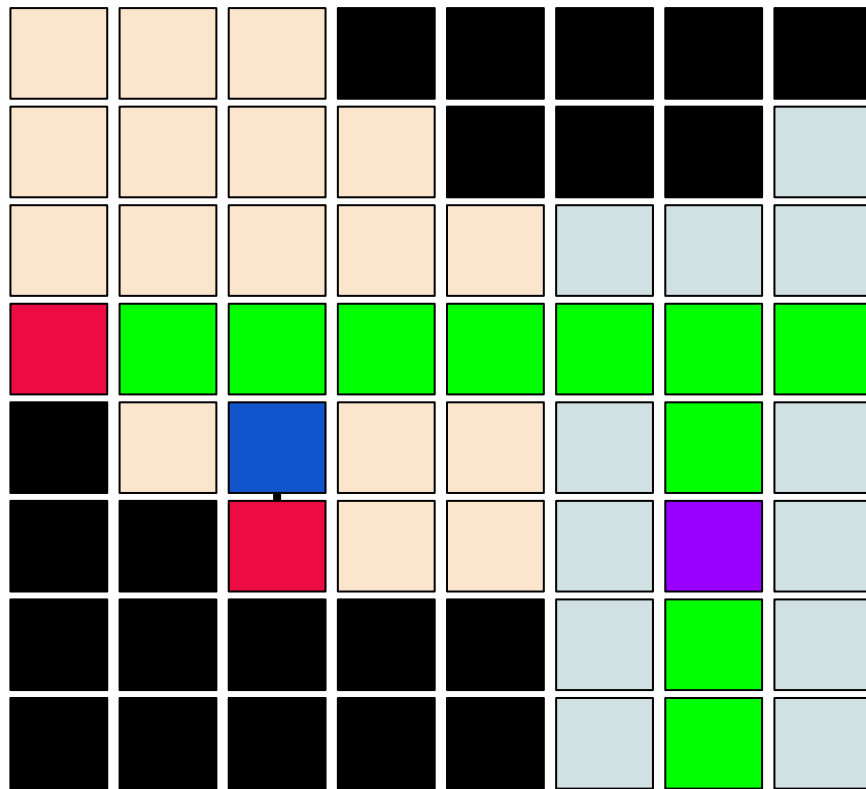
Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД



Модификация алгоритма Дейкстры – Алгоритм А*.

Алгоритм A*.

1. Временные метки M, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

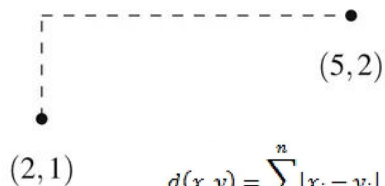
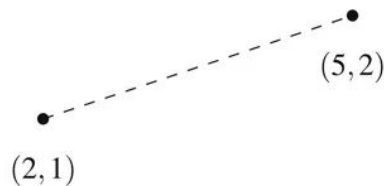
Напасть на врага – 10 ОД;

Посетить – 1 ОД

Модификация:

Добавляется эвристическая функция

Например, **Манхэттенское расстояние**



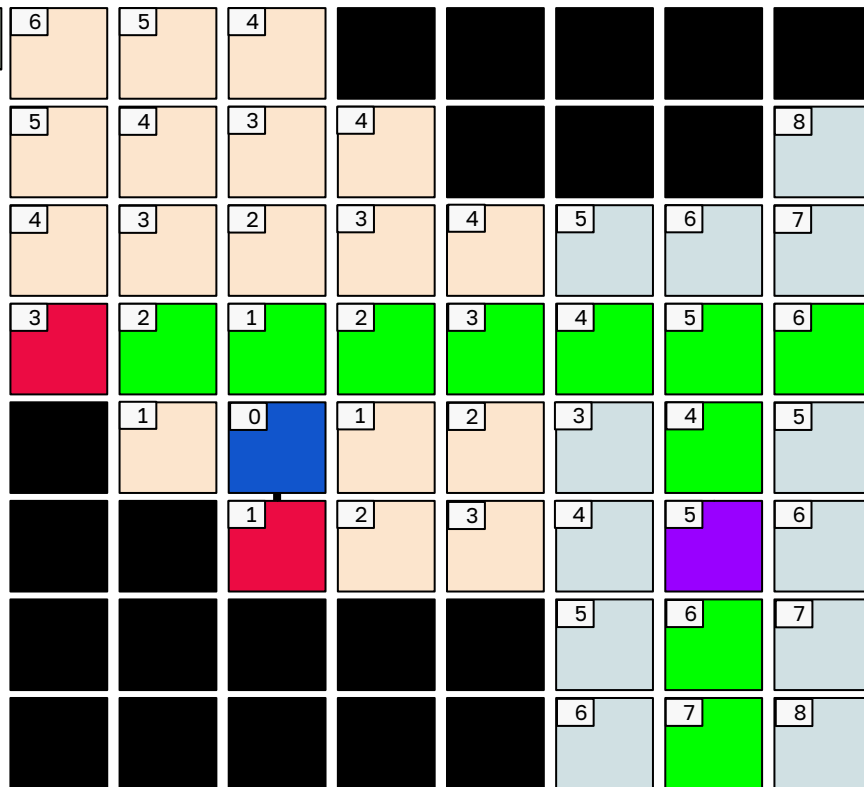
$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

2) Новая BM = мин (прошлая BM, значение в рассматриваемой вершине + значение ребра + значение ЭФ)

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки M, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

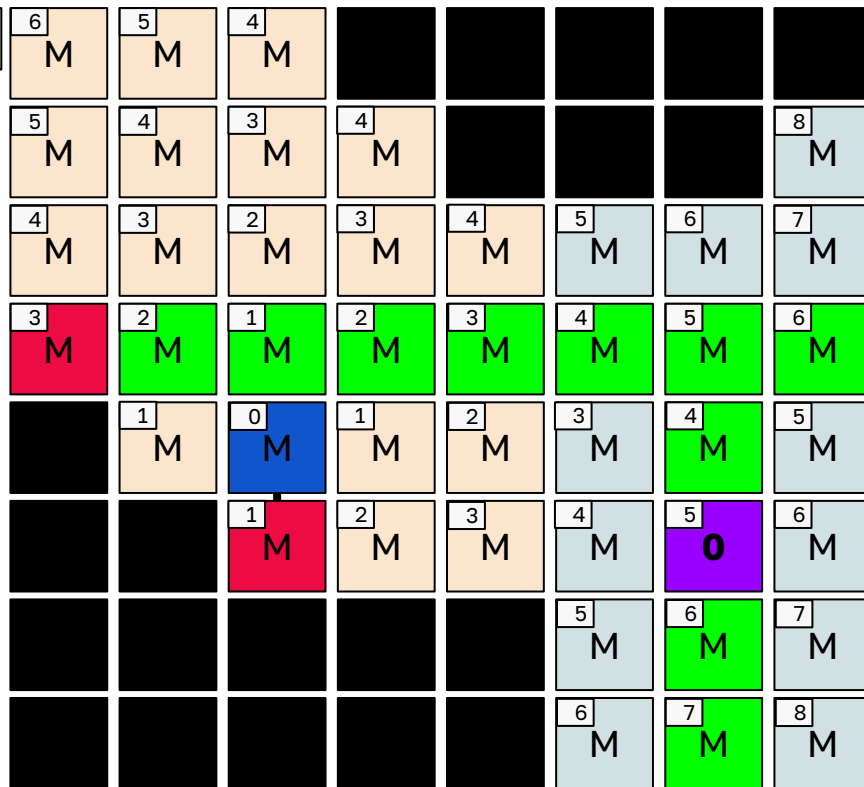
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

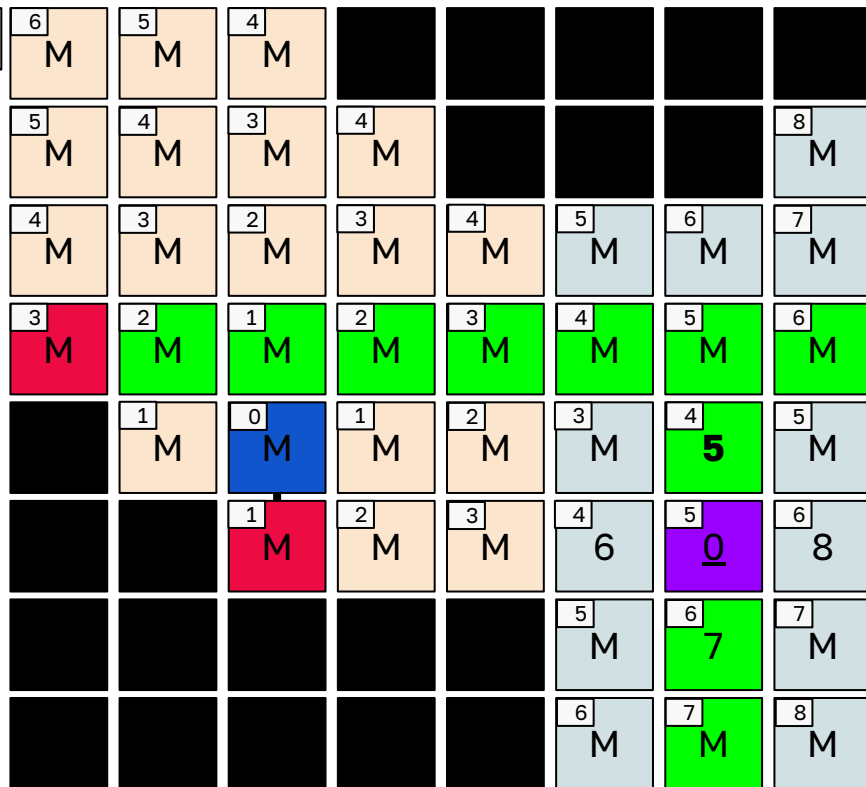
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

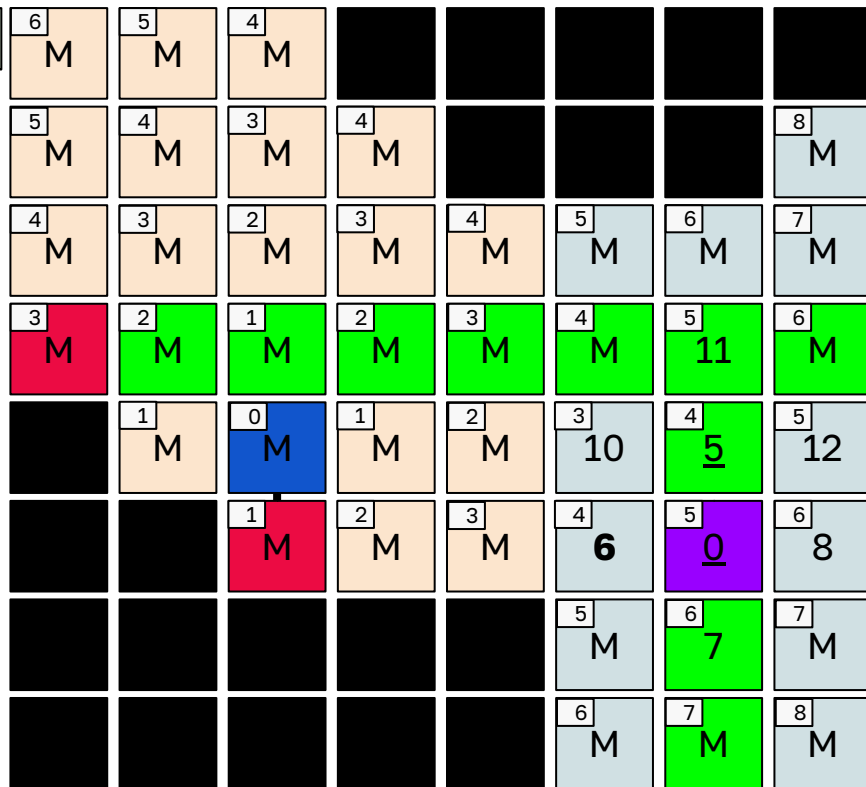
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

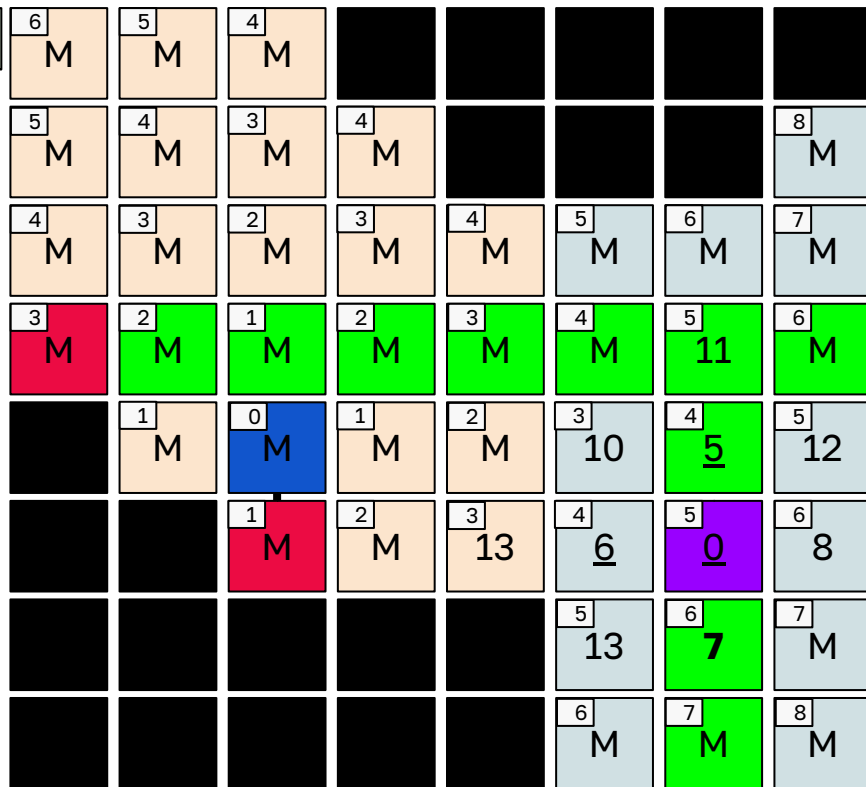
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

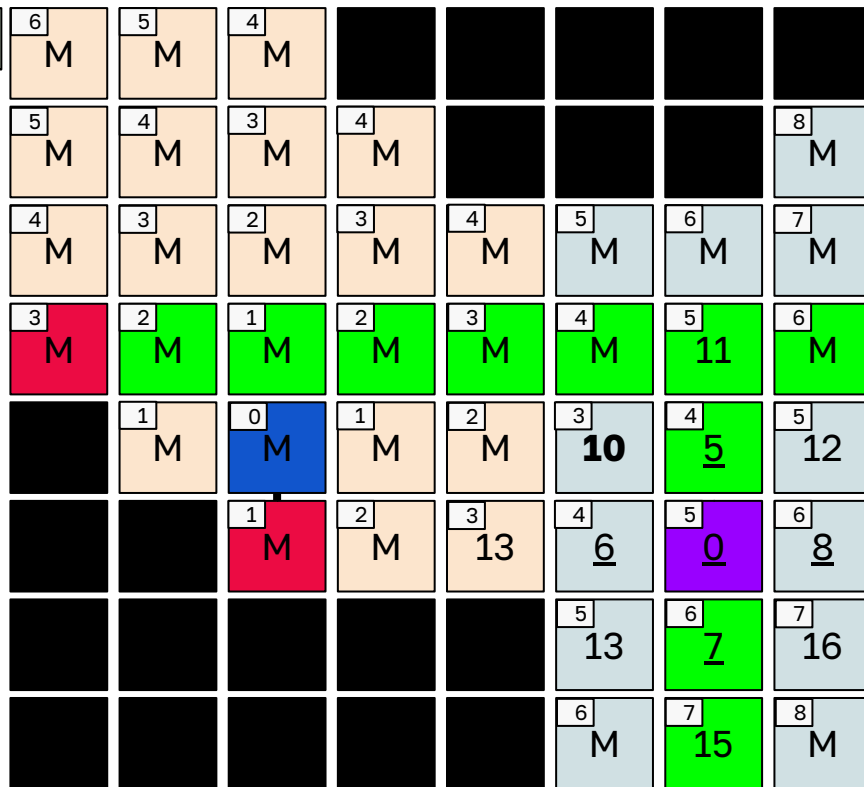
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

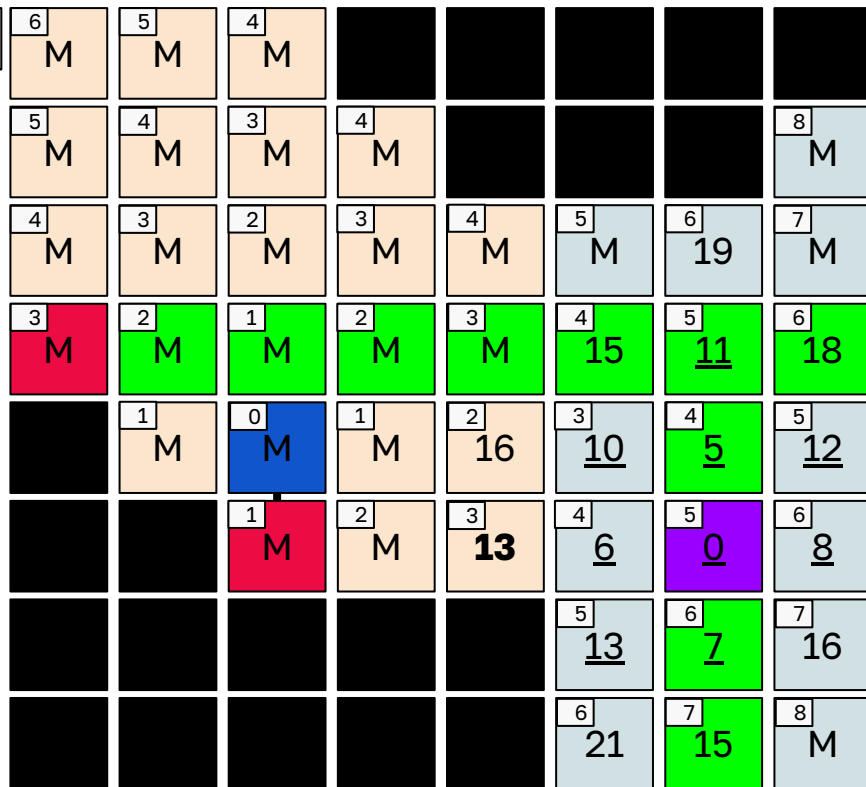
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до конечной точки

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

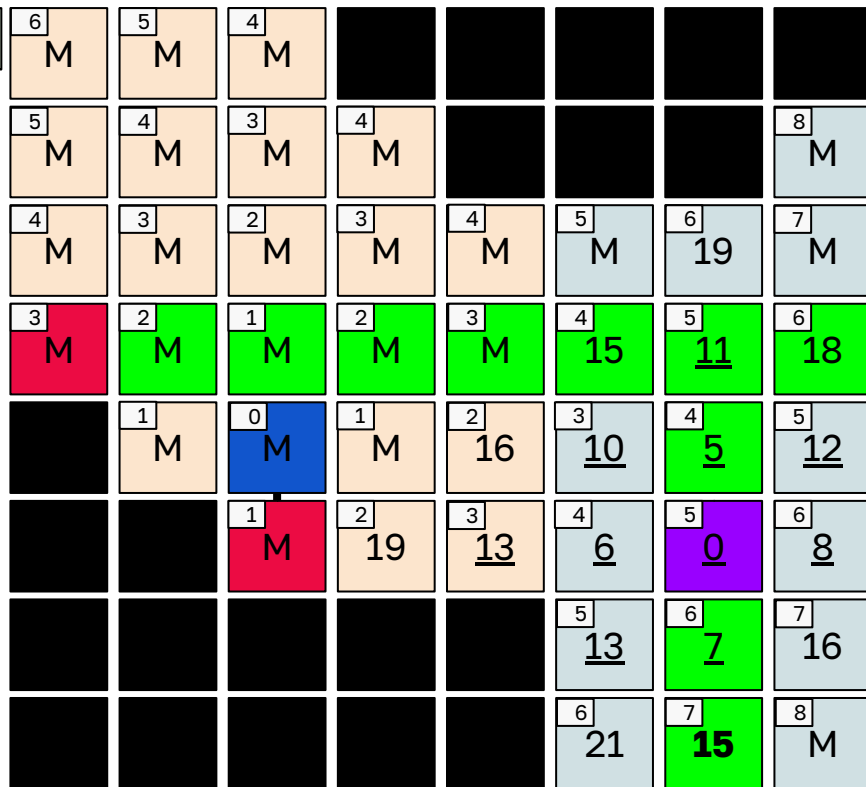
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

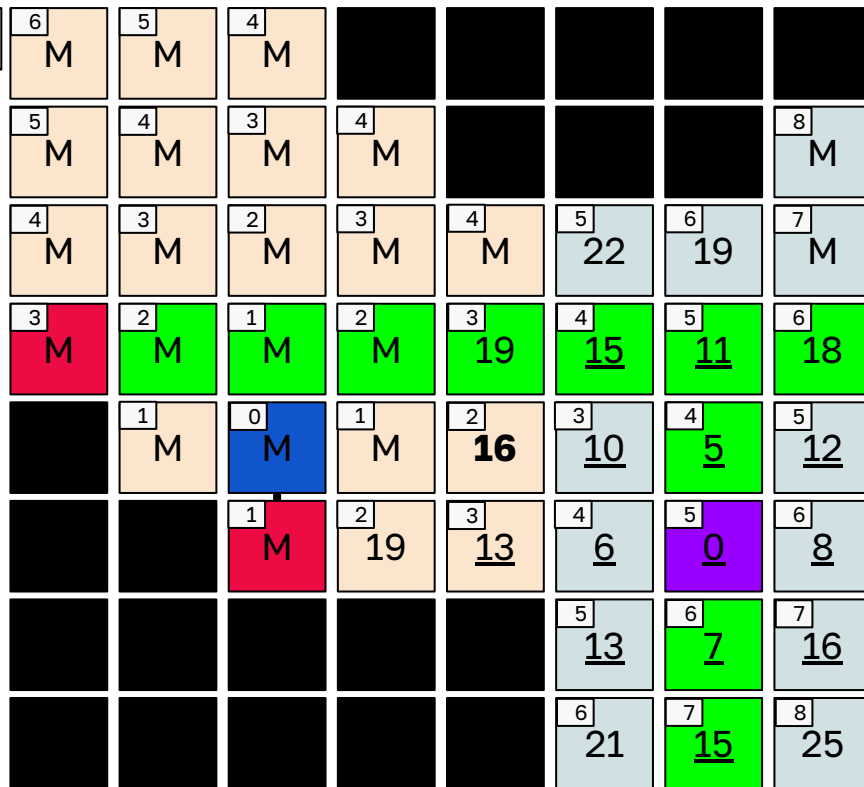
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

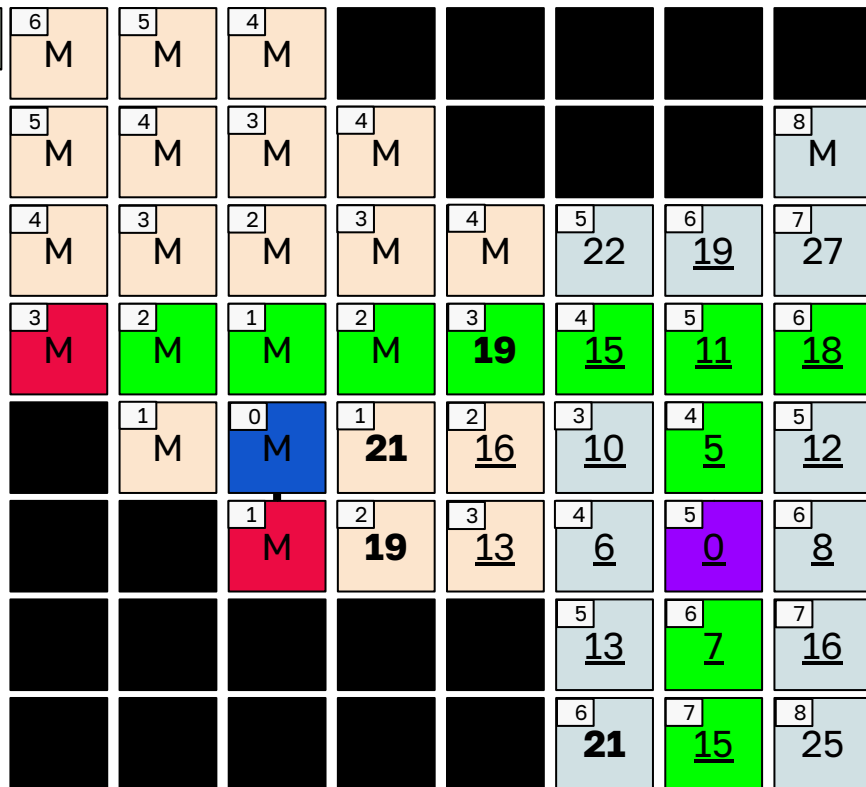
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

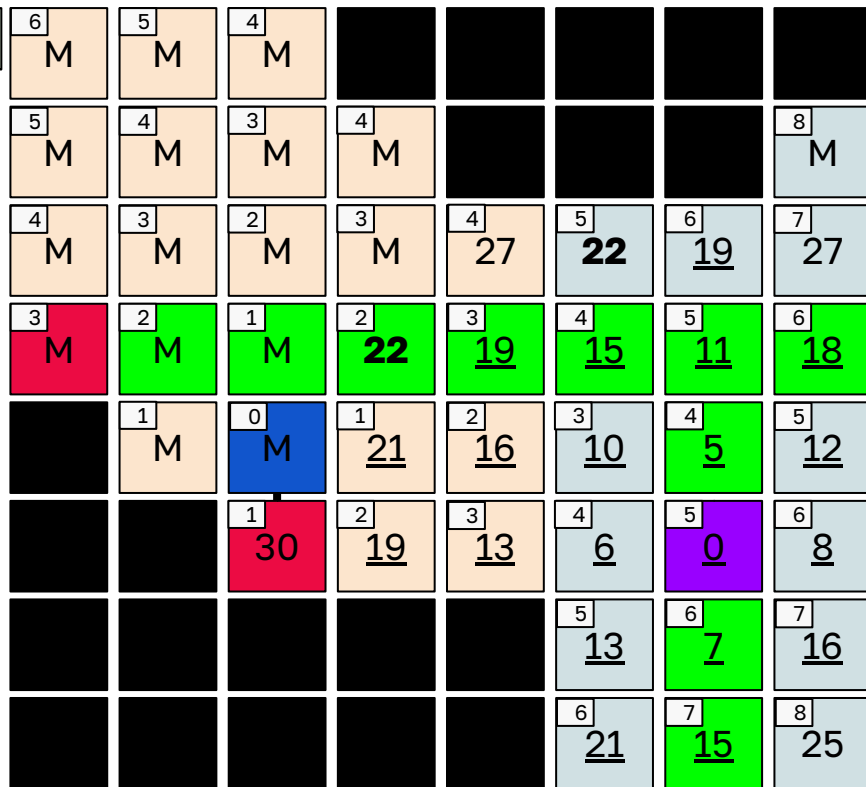
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

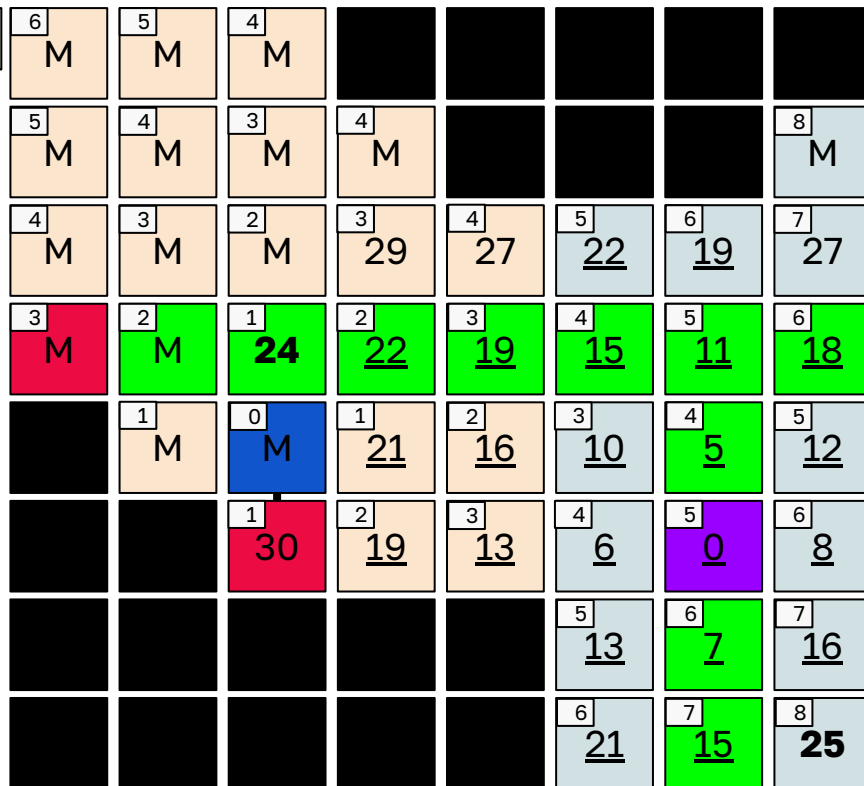
Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

⁶ М	⁵ М	⁴ М					
⁵ М	⁴ М	³ М	⁴ М				⁸ М
⁴ М	³ М	² 30	³ 29	⁴ 27	⁵ <u>22</u>	⁶ <u>19</u>	⁷ 27
³ М	² 27	¹ <u>24</u>	² <u>22</u>	³ <u>19</u>	⁴ <u>15</u>	⁵ <u>11</u>	⁶ <u>18</u>
	¹ М	⁰ М	¹ <u>21</u>	² <u>16</u>	³ <u>10</u>	⁴ 5	⁵ <u>12</u>
		¹ 30	² <u>19</u>	³ <u>13</u>	⁴ <u>6</u>	⁵ 0	⁶ <u>8</u>
					⁵ <u>13</u>	⁶ 7	⁷ <u>16</u>
					⁶ <u>21</u>	⁷ 15	⁸ <u>25</u>

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

⁶ М	⁵ М	⁴ М					
⁵ М	⁴ М	³ М	⁴ М				⁸ 37
⁴ М	³ 34	² 30	³ 29	⁴ <u>27</u>	⁵ <u>22</u>	⁶ <u>19</u>	⁷ <u>27</u>
³ 40	² <u>27</u>	¹ <u>24</u>	² <u>22</u>	³ <u>19</u>	⁴ <u>15</u>	⁵ <u>11</u>	⁶ <u>18</u>
	¹ 32	⁰ М	¹ <u>21</u>	² <u>16</u>	³ <u>10</u>	⁴ <u>5</u>	⁵ <u>12</u>
		¹ 30	² <u>19</u>	³ <u>13</u>	⁴ <u>6</u>	⁵ <u>0</u>	⁶ <u>8</u>
					⁵ <u>13</u>	⁶ <u>7</u>	⁷ <u>16</u>
					⁶ <u>21</u>	⁷ <u>15</u>	⁸ <u>25</u>

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

⁶ М	⁵ М	⁴ М					
⁵ М	⁴ М	³ 37	⁴ 37				⁸ 37
⁴ М	³ 34	² <u>30</u>	³ <u>29</u>	⁴ <u>27</u>	⁵ <u>22</u>	⁶ <u>19</u>	⁷ <u>27</u>
³ 40	² <u>27</u>	¹ <u>24</u>	² <u>22</u>	³ <u>19</u>	⁴ <u>15</u>	⁵ <u>11</u>	⁶ <u>18</u>
	¹ 32	⁰ 31	¹ <u>21</u>	² <u>16</u>	³ <u>10</u>	⁴ <u>5</u>	⁵ <u>12</u>
		¹ <u>30</u>	² <u>19</u>	³ <u>13</u>	⁴ <u>6</u>	⁵ <u>0</u>	⁶ <u>8</u>
					⁵ <u>13</u>	⁶ <u>7</u>	⁷ <u>16</u>
					⁶ <u>21</u>	⁷ <u>15</u>	⁸ <u>25</u>

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

⁶ М	⁵ М	⁴ М					
⁵ М	⁴ М	³ 37	⁴ 37				⁸ 37
⁴ М	³ 34	² <u>30</u>	³ <u>29</u>	⁴ <u>27</u>	⁵ <u>22</u>	⁶ <u>19</u>	⁷ <u>27</u>
³ 40	² <u>27</u>	¹ <u>24</u>	² <u>22</u>	³ <u>19</u>	⁴ <u>15</u>	⁵ <u>11</u>	⁶ <u>18</u>
	¹ 32	⁰ 31	¹ <u>21</u>	² <u>16</u>	³ <u>10</u>	⁴ <u>5</u>	⁵ <u>12</u>
		¹ <u>30</u>	² <u>19</u>	³ <u>13</u>	⁴ <u>6</u>	⁵ <u>0</u>	⁶ <u>8</u>
					⁵ <u>13</u>	⁶ <u>7</u>	⁷ <u>16</u>
					⁶ <u>21</u>	⁷ <u>15</u>	⁸ <u>25</u>

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: Манхэттенское расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм A*.

1. Временные метки M, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины
(временные метки)
3. Фиксируем обработанную вершину
(временная метка → постоянная)

Примеры эвристик

Поведение алгоритма сильно зависит от того, какая эвристика используется. В свою очередь, выбор эвристики зависит от постановки задачи. Часто A* используется для моделирования перемещения по поверхности, покрытой координатной сеткой.

- Если мы можем перемещаться в четырех направлениях, то в качестве эвристики стоит выбрать манхэттенское расстояние^[1]

$$h(v) = |v.x - goal.x| + |v.y - goal.y|.$$

- Расстояние Чебышева^[2] применяется, когда к четырем направлениям добавляются диагонали:

$$h(v) = \max(|v.x - goal.x|, |v.y - goal.y|).$$

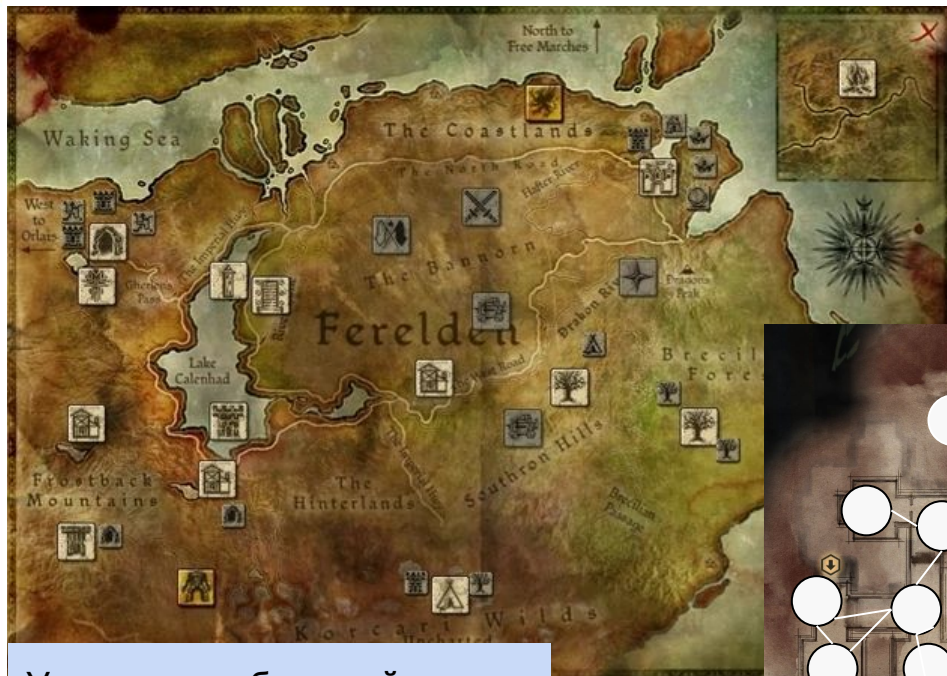
- Если передвижение не ограничено сеткой, то можно использовать евклидово расстояние по прямой:

$$h(v) = \sqrt{(v.x - goal.x)^2 + (v.y - goal.y)^2}.$$

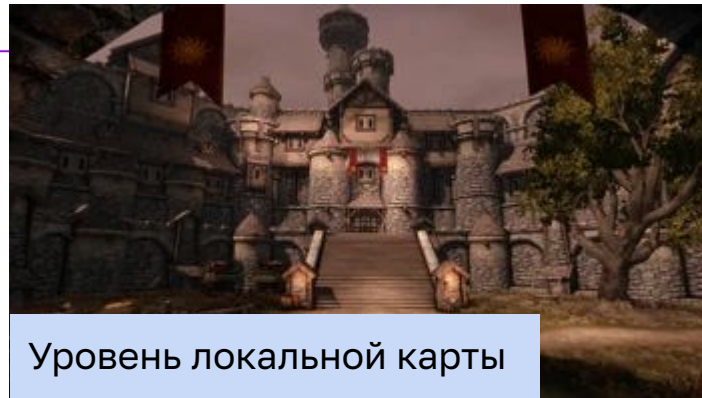
Также стоит обратить внимание на то как соотносятся $f(v)$ и $h(v)$. Если они измеряются в разных величинах (например, $g(v)$ — это расстояние в километрах, а $h(v)$ — оценка времени пути в часах) A* может выдать некорректный результат.

ГЛАВА 3 _ Навигация

ІІТМО



Уровень глобальной карты



Уровень локальной карты

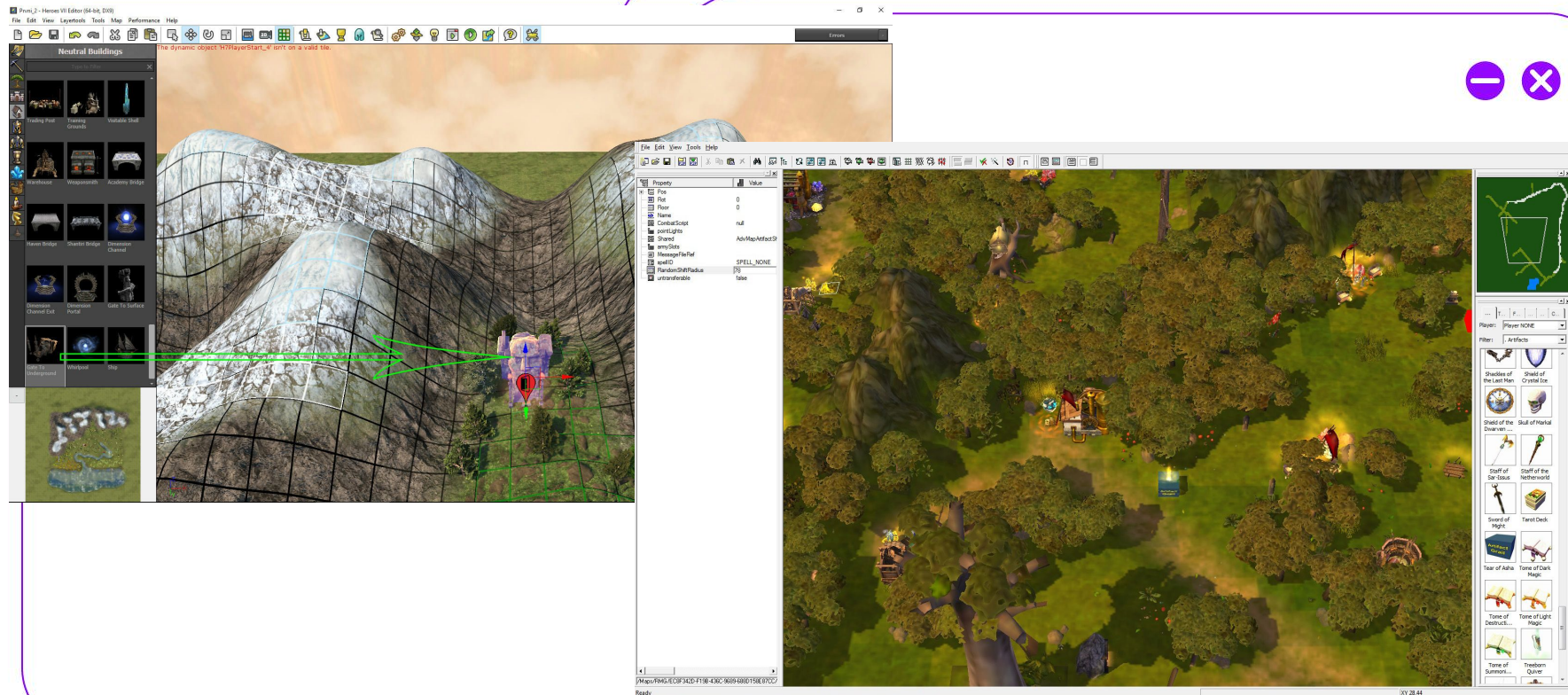


Уровень Блоков



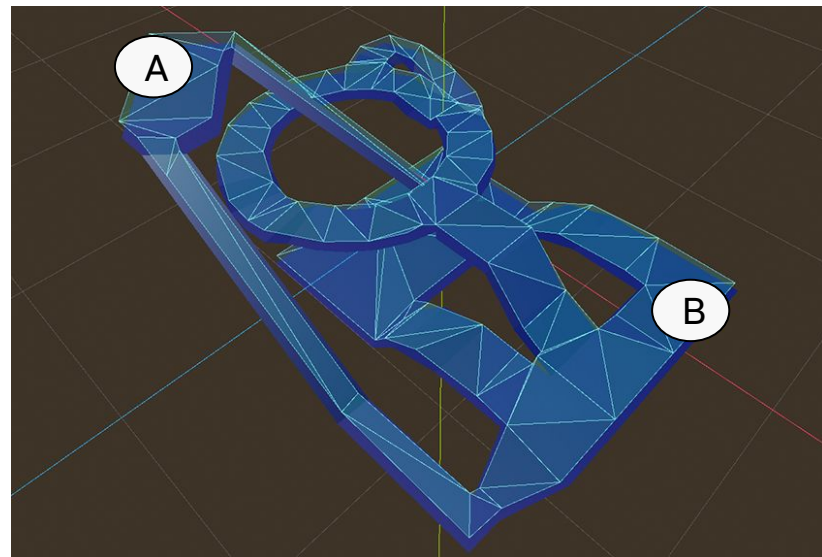
Уровень пути

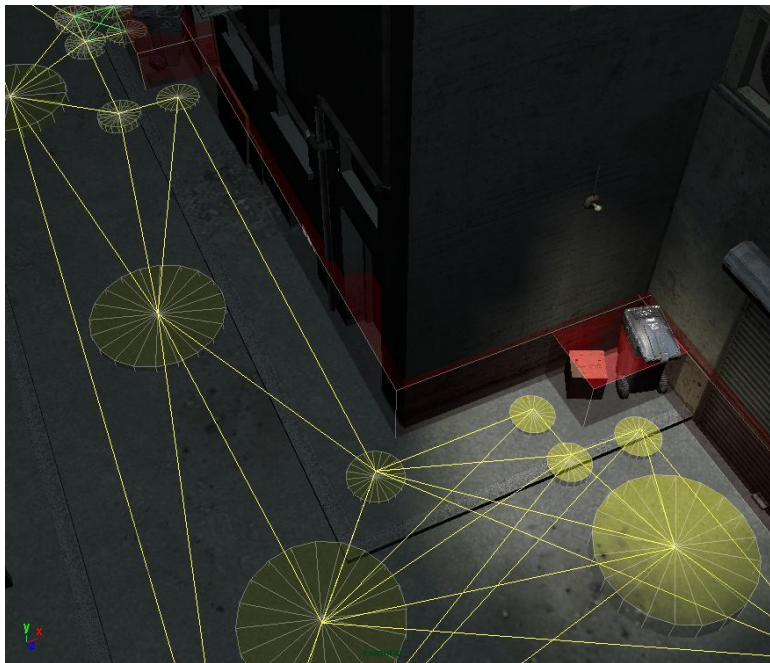
ГЛАВА 3 _ Навигация



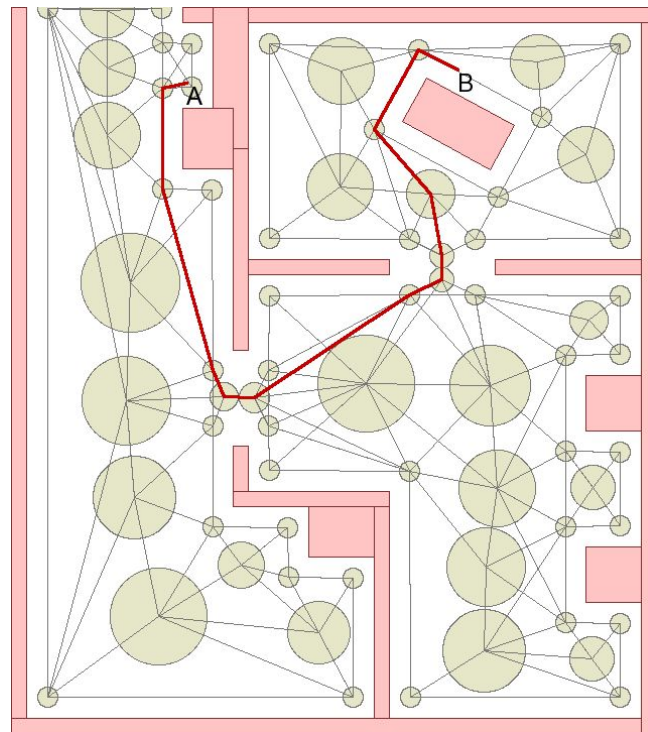
Навигационный мэш (navigation mesh), также известный как навигационная сетка, является одной из популярных техник для реализации навигации в играх с искусственным интеллектом (ИИ). Он представляет собой графическое представление игрового мира, разбитого на множество маленьких участков, называемых полигональными ячейками или треугольниками. Каждая ячейка описывает площадь проходимости или непроходимости для персонажей или объектов в игре.

Навигационный мэш используется для определения пути движения персонажей ИИ. Он позволяет оптимизировать вычисление путей, исключая непроходимые участки или уменьшая количество проверок столкновений. К примеру, персонаж может перемещаться по поверхности полигональной ячейки или находить ближайший путь к цели, используя информацию из навигационного мэша.





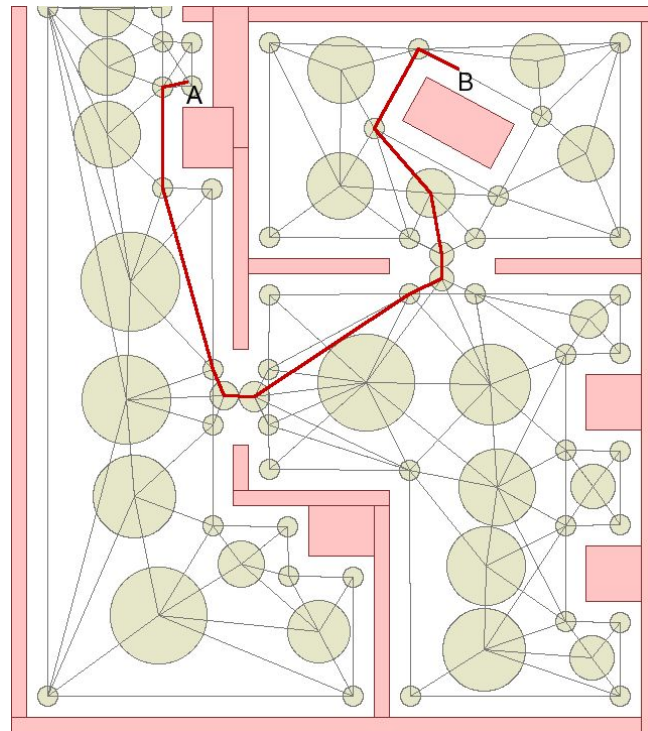
[Полезная статья про прикладную навигацию в играх](#)



1. Граф может **легко добавлять** в себя навигационные вершины объектов, размещенных в окружающей среде.
2. Граф **не привязан к геометрии** уровня (как навмеш), он может обеспечить дополнительную гибкость при проектировании уровня. Узлы обычно представлены в виде объектов в инструментах проектирования уровней. Как таковые, ими легче манипулировать, чем треугольниками в навигационной сетке, где дизайнеры должны заботиться о сохранении **топологических свойств** сетки.



[Полезная статья про прикладную навигацию в играх](#)

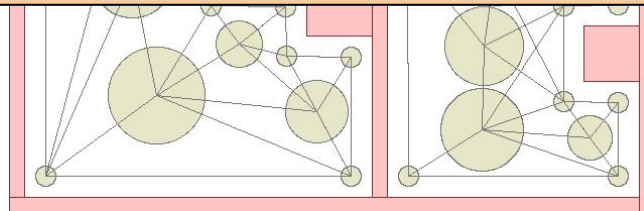


1. Граф может **легко добавлять** в себя навигационные вершины объектов, размещенных в окружающей среде.
2. Граф **не привязан к геометрии** уровня (как навмеш), он может обеспечить дополнительную гибкость при проектировании уровня. Узлы обычно представлены в виде объектов в инструментах проектирования уровней. Как таковые, ими легче манипулировать, чем треугольниками в навигационной сетке, где дизайнеры должны заботиться о сохранении **топологических свойств** сетки.



[Полезная статья про прикладную навигацию в играх](#)

1. Ручное создание навигационного графа и его настройка могут занять **очень много времени**.
2. Графы, созданные вручную, **трудно обновлять**, если геометрия меняется в процессе проектирования, и они требуют особой осторожности в случае изменений во время выполнения в игре.



1. Граф может **легко добавлять** в себя навигационные вершины объектов, размещенных в окружающей среде.
2. Граф **не привязан к геометрии** уровня (как навмеш), он может обеспечить дополнительную гибкость при проектировании уровня. Узлы обычно представлены в виде объектов в инструментах проектирования уровней. Как таковые, ими легче манипулировать, чем треугольниками в навигационной сетке, где дизайнеры должны заботиться о сохранении **топологических свойств** сетки.

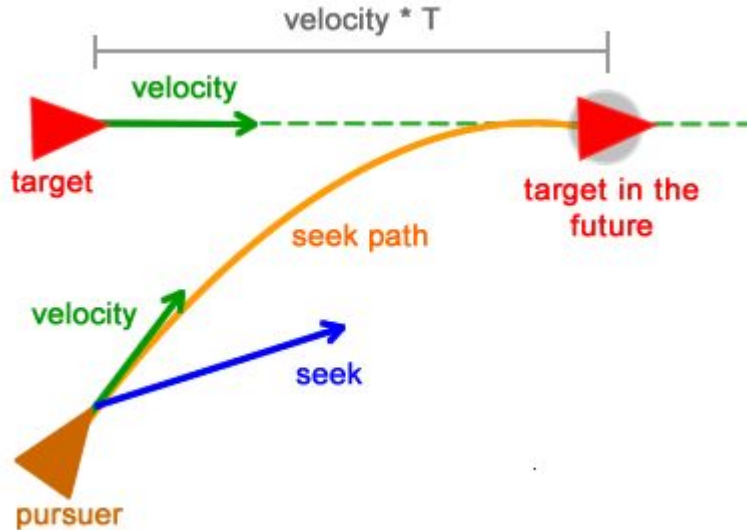
1. Ручное создание навигационного графа и его настройка могут занять **очень много времени**.
2. Графы, созданные вручную, **трудно обновлять**, если геометрия меняется в процессе проектирования, и они требуют особой осторожности в случае изменений во время выполнения в игре.

Выводы:

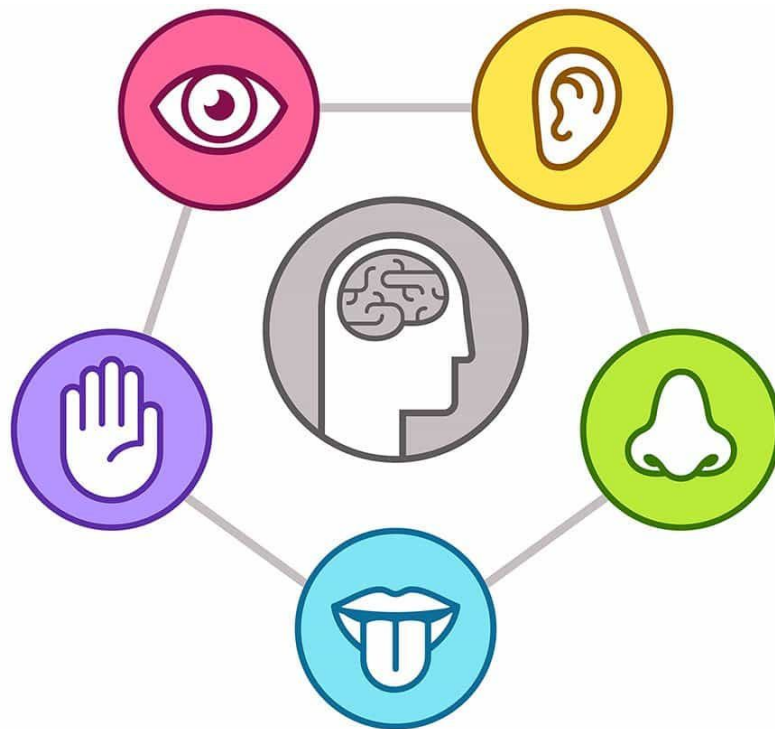
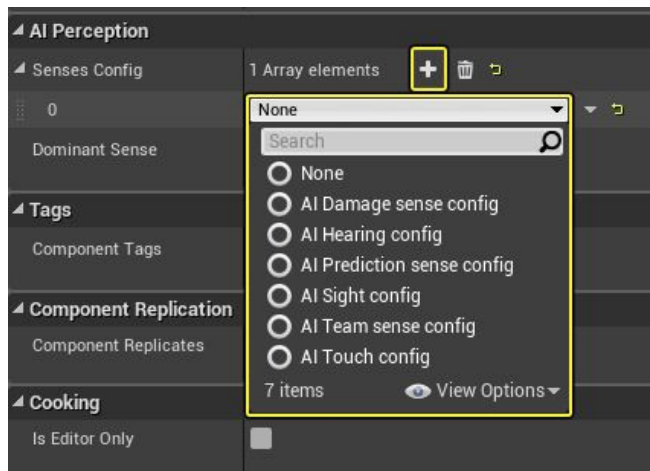
Навигационная сетка (навмеш) более уместна, когда навигационные данные генерируются автоматически, а навигационный граф может быть более предпочтительным, когда требуется ввод данных левел дизайнеров. Это приводит к необходимости упрощения создания и обслуживания навигационного графа, когда он подлежит ручному изменению.

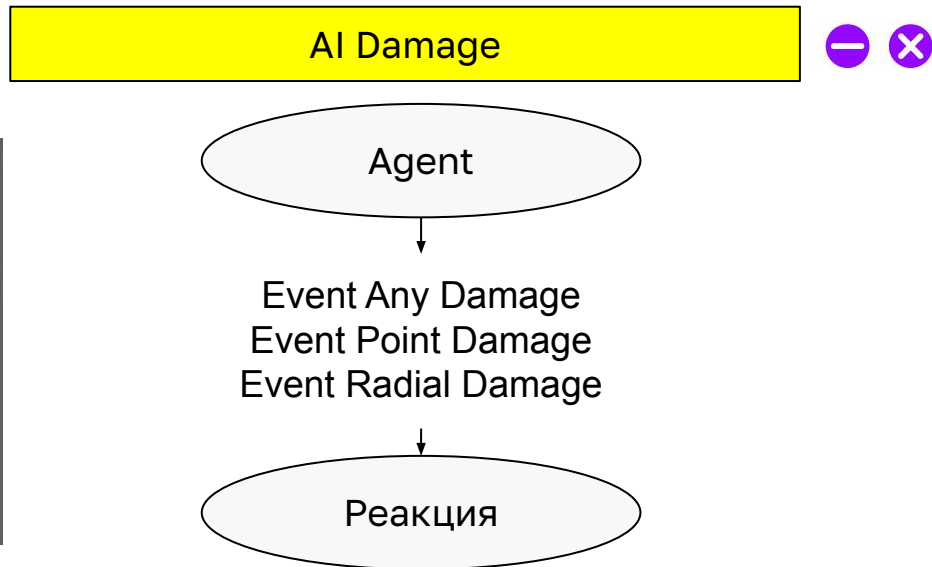
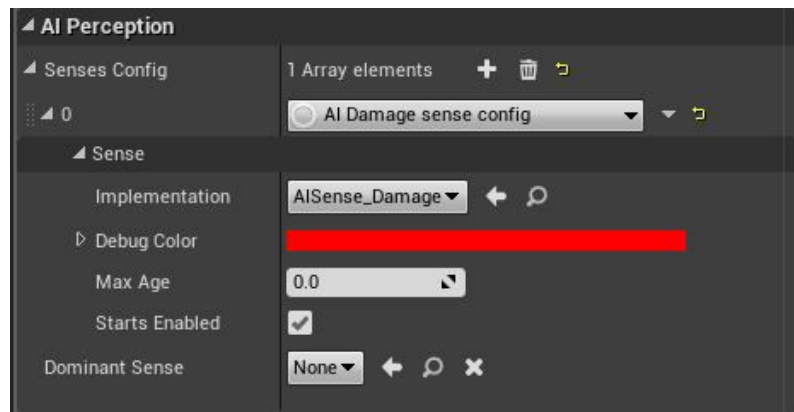
[Полезная статья про прикладную навигацию в играх](#)

Еще чуть-чуть про движение, которое мы не будем покрывать в лекциях, но и не сказать – нельзя.



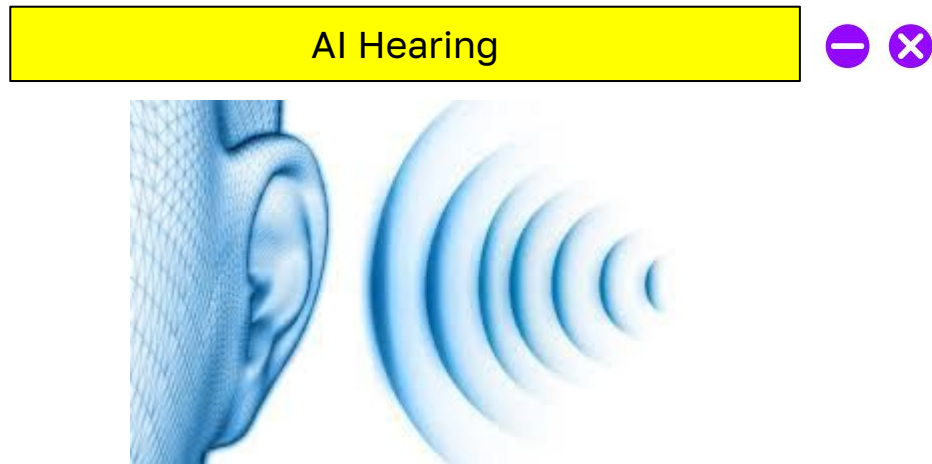
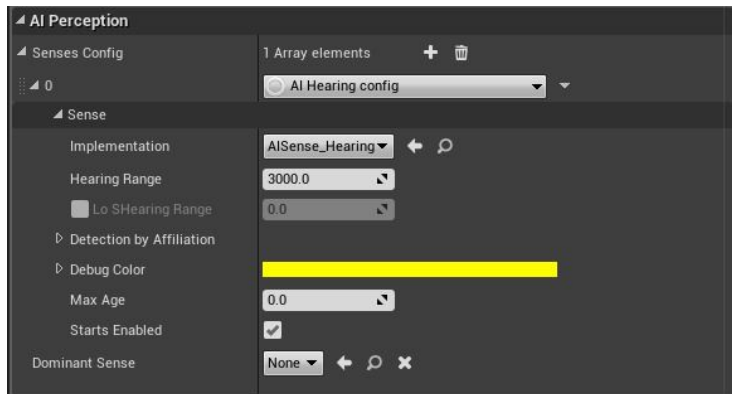






Специфические параметры:

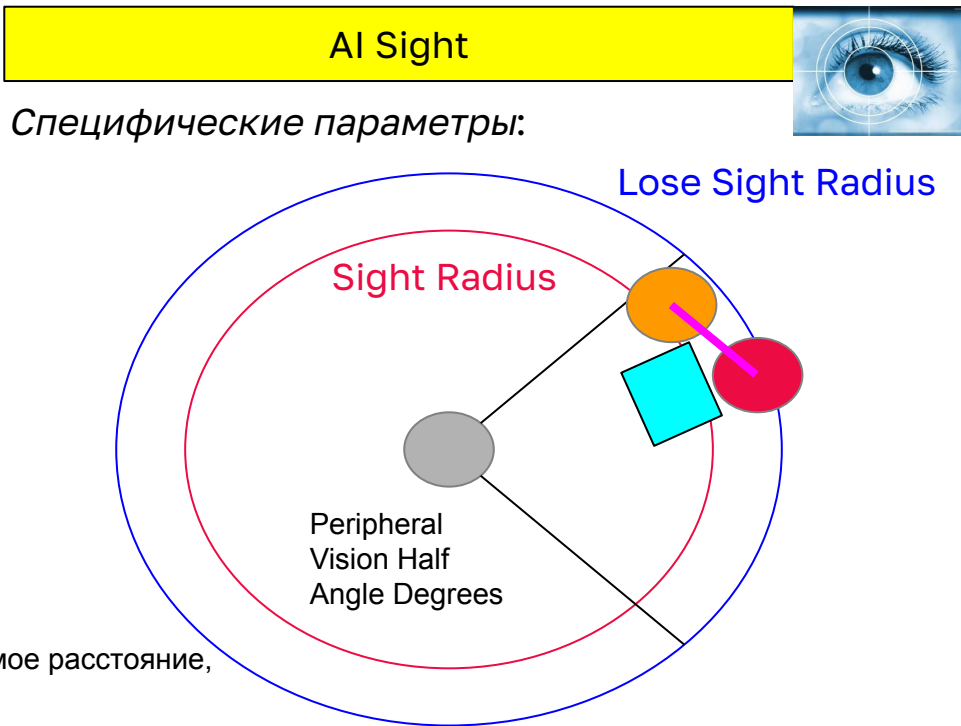
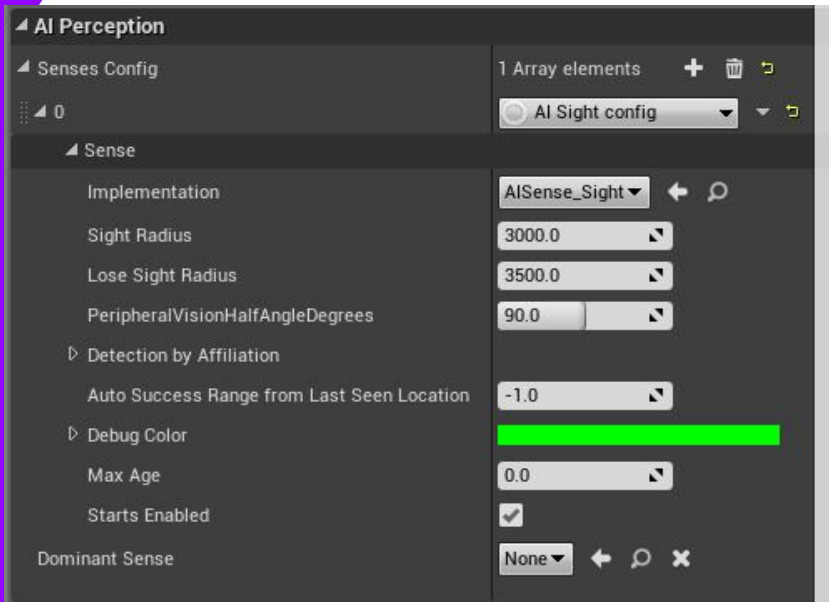
max age - Определяет продолжительность, в течение которой стимулы, генерируемые этим чувством, забываются (0 означает, что они никогда не забываются).



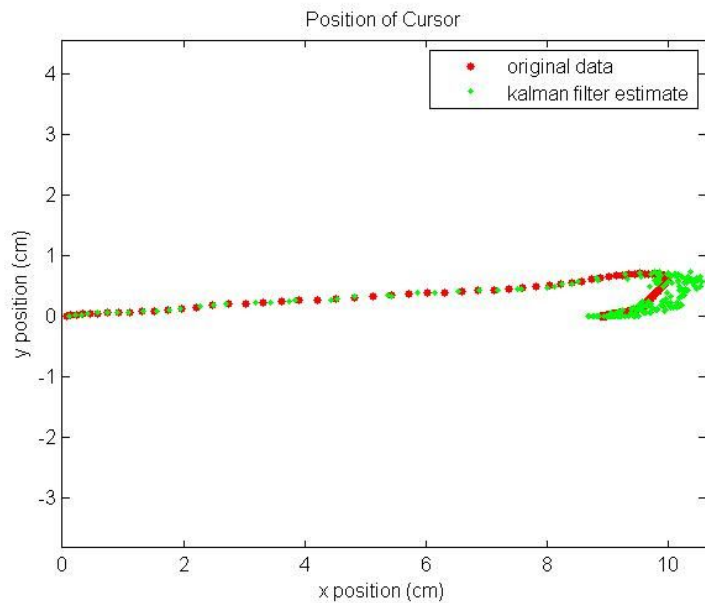
Специфические параметры:

Hearing Range - Диапазон слышимости

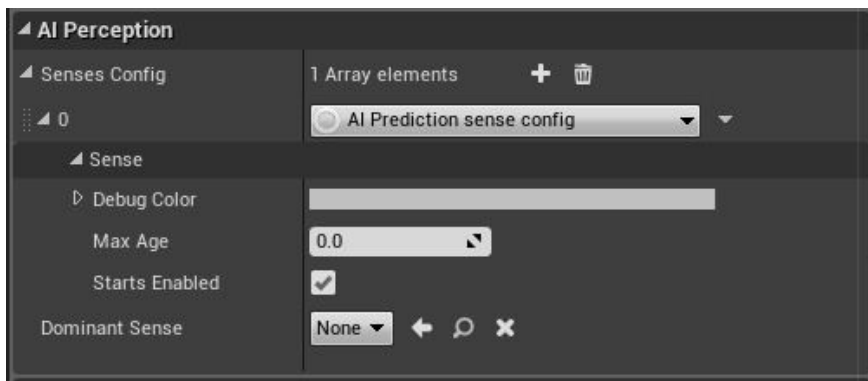
Detection by Affiliation - Определяет, могут ли враги, нейтралы или товарищеские матчи вызвать это чувство



Auto Success Range from Last Seen Location - допустимое расстояние, чтобы агент помнил ("знал") о нахождении сущности.



AI Prediction



ГЛАВА 3 _ Технологии программирования ИИИ

Знания

ІІТМО



Тэги

ГЛАВА 3 _ Технологии программирования ИИИ

Знания

ИТМО



Тэги



Умные объекты

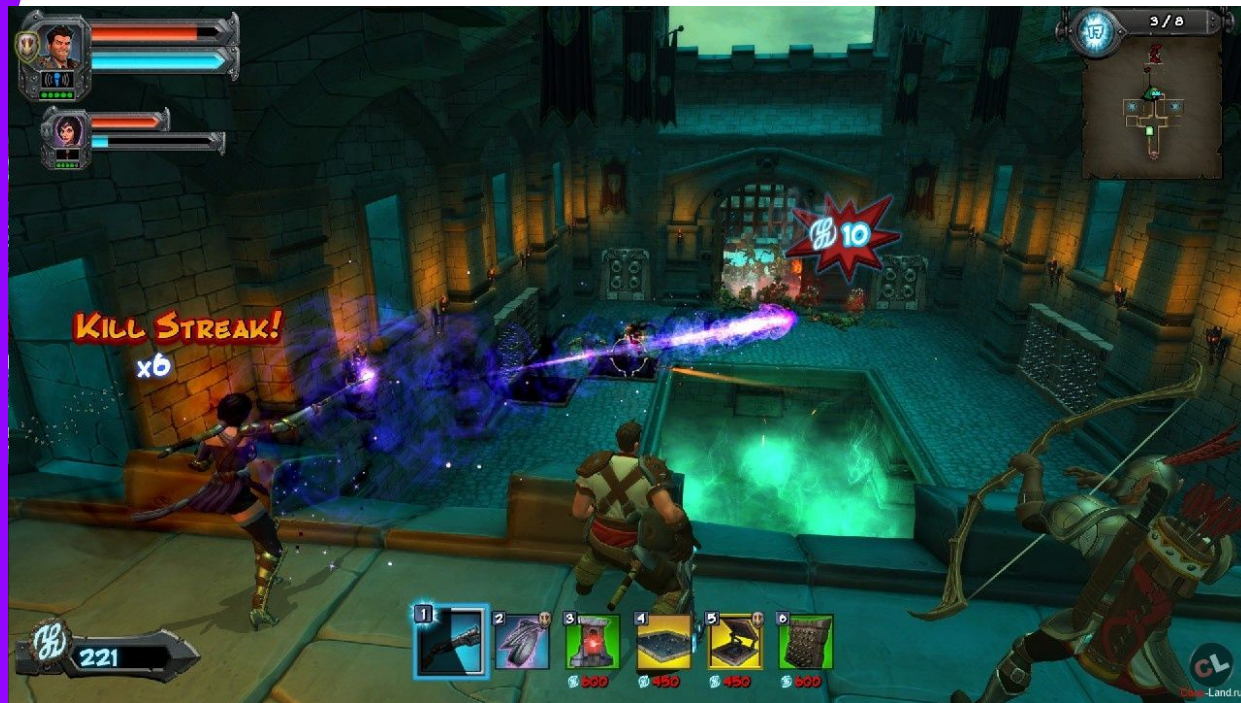
ГЛАВА 3 _ Технологии программирования ИИИ

Знания

ИТМО



Умные объекты



Лучник AI



```
{  
  "count_enemies":3,  
  "type_enemy":"simple orc",  
  "distance":120,  
  "ally":true  
}
```

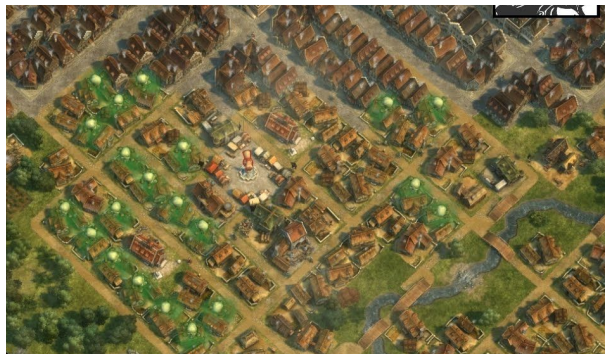
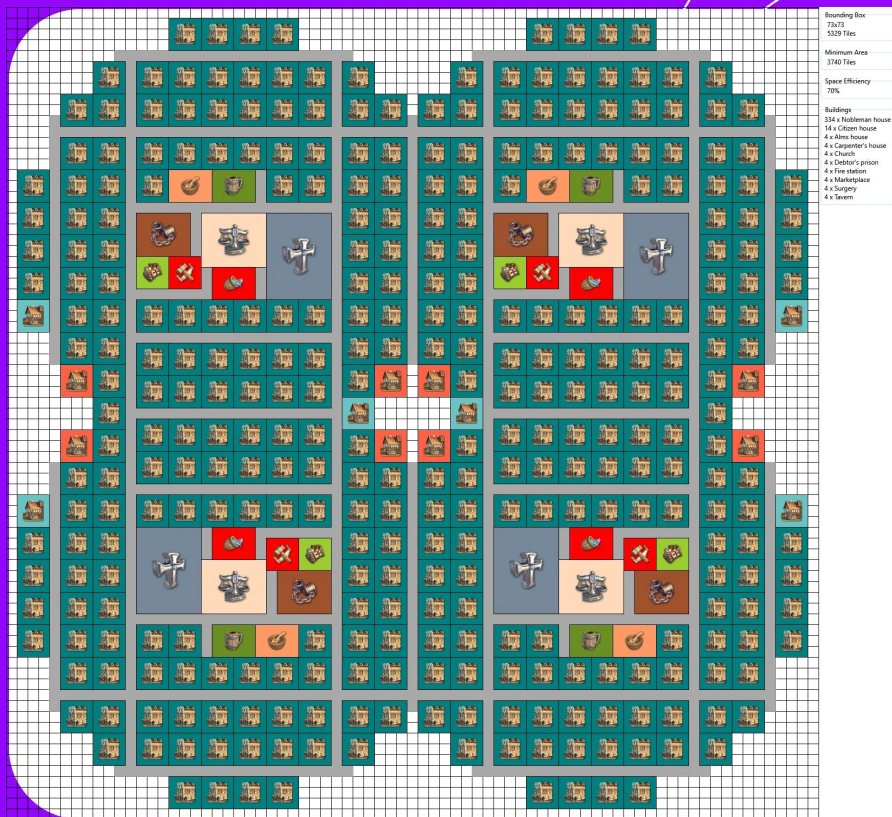
Dynamic blackboard

Static blackboard



ГЛАВА 3 _ Технологии программирования ИИИ

Influence maps



EQS (Environment Query System) - это система запросов к окружению, которая используется в игровом ИИ для принятия решений на основе информации об окружающем мире. EQS предоставляет возможность игровому ИИ задавать вопросы о своей среде и получать соответствующие ответы в виде структурированных данных.



EQS (Environment Query System) - это система запросов к окружению, которая используется в игровом ИИ для принятия решений на основе информации об окружающем мире. EQS предоставляет возможность игровому ИИ задавать вопросы о своей среде и получать соответствующие ответы в виде структурированных данных.



EQS может использоваться для решения различных задач в игровом ИИ, таких как поиск и выбор целей, планирование пути, расчеты столкновений и т.д. Она позволяет ИИ осуществлять анализ окружающей среды и принимать решения на основе полученной информации.

EQS (Environment Query System) - это система запросов к окружению, которая используется в игровом ИИ для принятия решений на основе информации об окружающем мире. EQS предоставляет возможность игровому ИИ задавать вопросы о своей среде и получать соответствующие ответы в виде структурированных данных.



EQS может использоваться для решения различных задач в игровом ИИ, таких как поиск и выбор целей, планирование пути, расчеты столкновений и т.д. Она позволяет ИИ осуществлять анализ окружающей среды и принимать решения на основе полученной информации.

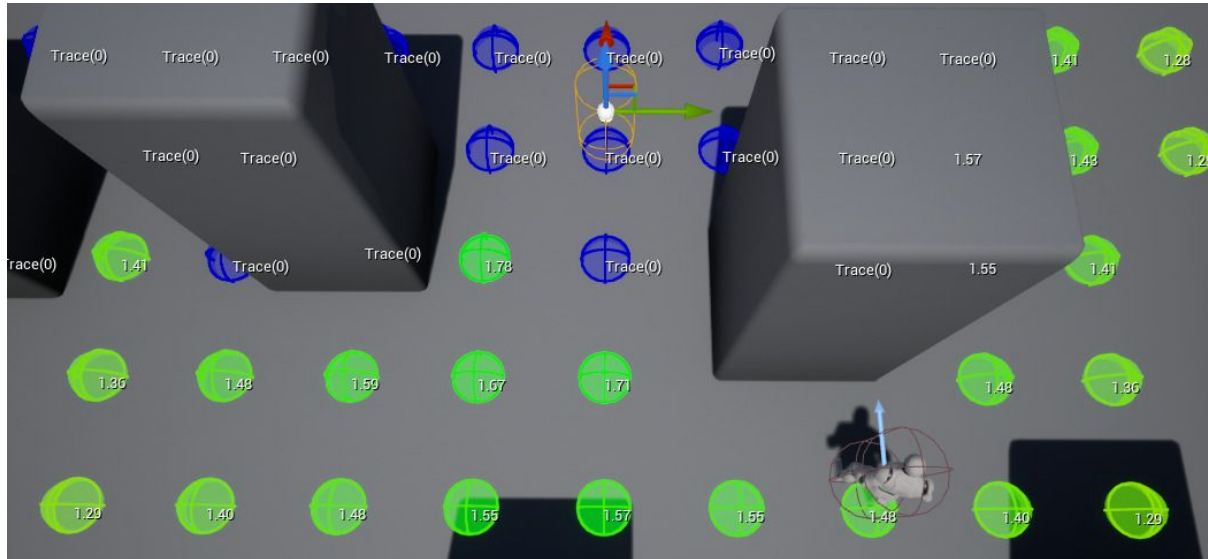
EQS работает на основе задания запросов в виде шаблонов, которые определяют критерии оценки окружения. Каждый шаблон содержит вопросы, которые должны быть заданы о среде, и формулу для оценки ответов на эти вопросы. Игровой ИИ может задать несколько шаблонов, и EQS будет применять их последовательно для получения наилучшего решения.

EQS (Environment Query System) - это система запросов к окружению, которая используется в игровом ИИ для принятия решений на основе информации об окружающем мире. EQS предоставляет возможность игровому ИИ задавать вопросы о своей среде и получать соответствующие ответы в виде структурированных данных.



Примером использования EQS может быть поиск наилучшей цели для атаки в игре. Игровой ИИ может создавать шаблоны, которые включают критерии, такие как расстояние до цели, ее здоровье, близость к союзникам и т.д. EQS будет оценивать каждую возможную цель по заданным критериям и вернет наилучший вариант для атаки.

EQS (Environment Query System) - это система запросов к окружению, которая используется в игровом ИИ для принятия решений на основе информации об окружающем мире. EQS предоставляет возможность игровому ИИ задавать вопросы о своей среде и получать соответствующие ответы в виде структурированных данных.



ГЛАВА 3 _ Планирование

ІТМО



ГЛАВА 3 _ Планирование



Goal-Oriented Action Planning (GOAP) – это методика планирования действий, которая широко применяется в разработке искусственного интеллекта для видеоигр. В GOAP акцент делается на определении иерархии целей и нахождении наиболее эффективного пути для достижения этих целей.

М

Goal-Oriented Action Planning (GOAP) – это методика планирования действий, которая широко применяется в разработке искусственного интеллекта для видеоигр. В GOAP акцент делается на определении иерархии целей и нахождении наиболее эффективного пути для достижения этих целей.

Процесс планирования в GOAP обычно состоит из **следующих шагов**:

1. Определение текущего состояния: ИИ анализирует текущую ситуацию, чтобы определить, **какие цели необходимо** достичь.
2. Определение возможных действий: ИИ анализирует доступные действия, которые **может выполнить**, чтобы **приблизиться** к достижению целей.
3. Планирование последовательности действий: ИИ определяет **порядок выполнения** выбранных действий, чтобы достичь цели.
4. Выполнение плана: ИИ следует определенному плану действий для достижения цели.

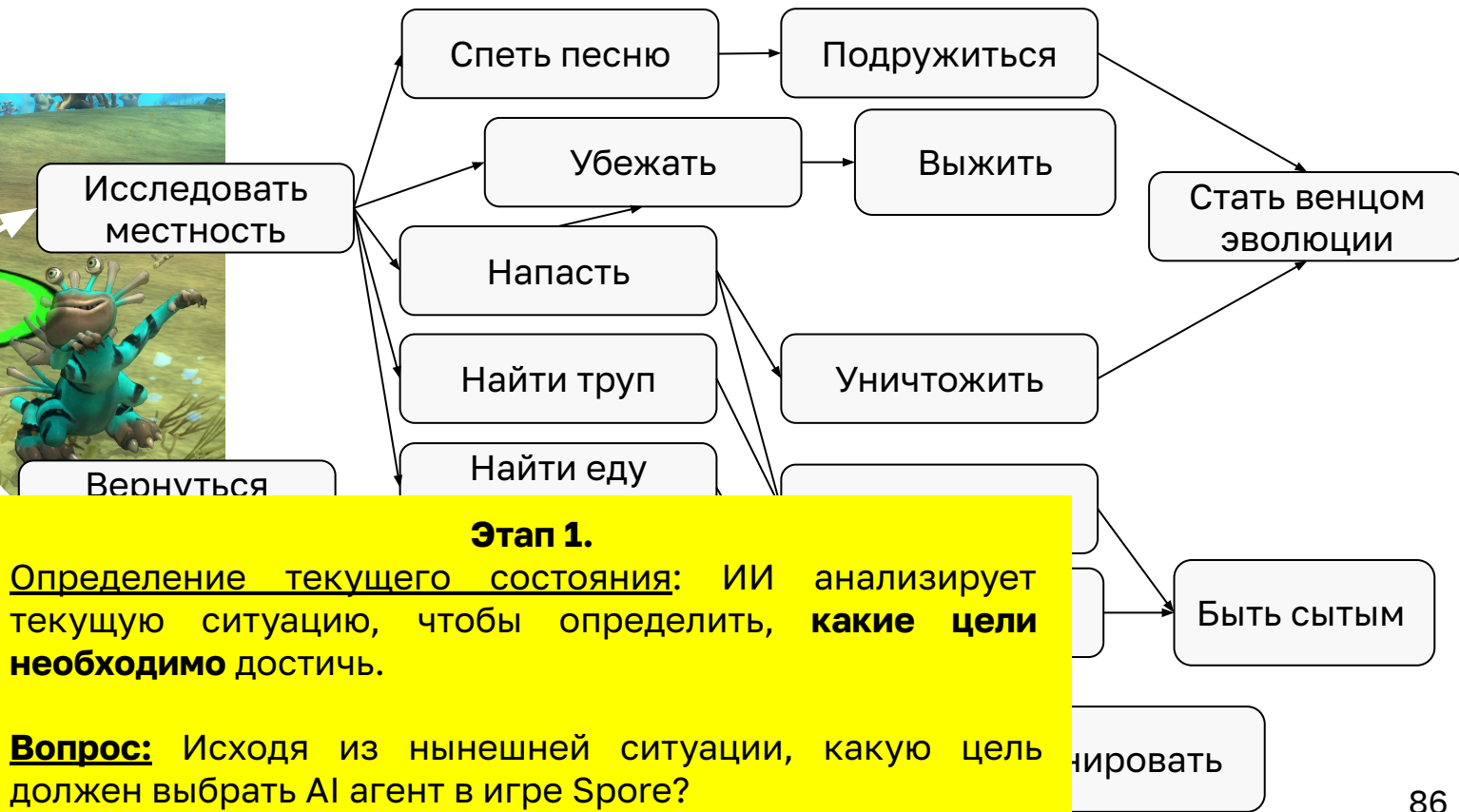
Goal-Oriented Action Planning (GOAP) – это методика планирования действий, которая широко применяется в разработке искусственного интеллекта для видеоигр. В GOAP акцент делается на определении иерархии целей и нахождении наиболее эффективного пути для достижения этих целей.

Процесс планирования в GOAP обычно состоит из **следующих шагов**:

1. Определение текущего состояния: ИИ анализирует текущую ситуацию, чтобы определить, **какие цели необходимо** достичь.
2. Определение возможных действий: ИИ анализирует доступные действия, которые **может выполнить**, чтобы **приблизиться** к достижению целей.
3. Планирование последовательности действий: ИИ определяет **порядок выполнения** выбранных действий, чтобы достичь цели.
4. Выполнение плана: ИИ следует определенному плану действий для достижения цели.

То есть в результате мы получаем список действий для достижения соответствующей ситуации цели.

ГЛАВА 3 _ Планирование



- Голод
- Выживание
- Эволюция
- Высшая цель

Этап 2.

Определение возможных действий:
ИИ анализирует доступные действия, которые **может выполнить**, чтобы **приблизиться** к достижению целей.

Планирование

ИТМО



- Голод
- Выживание
- Эволюция
- Высшая цель



Этап 3.

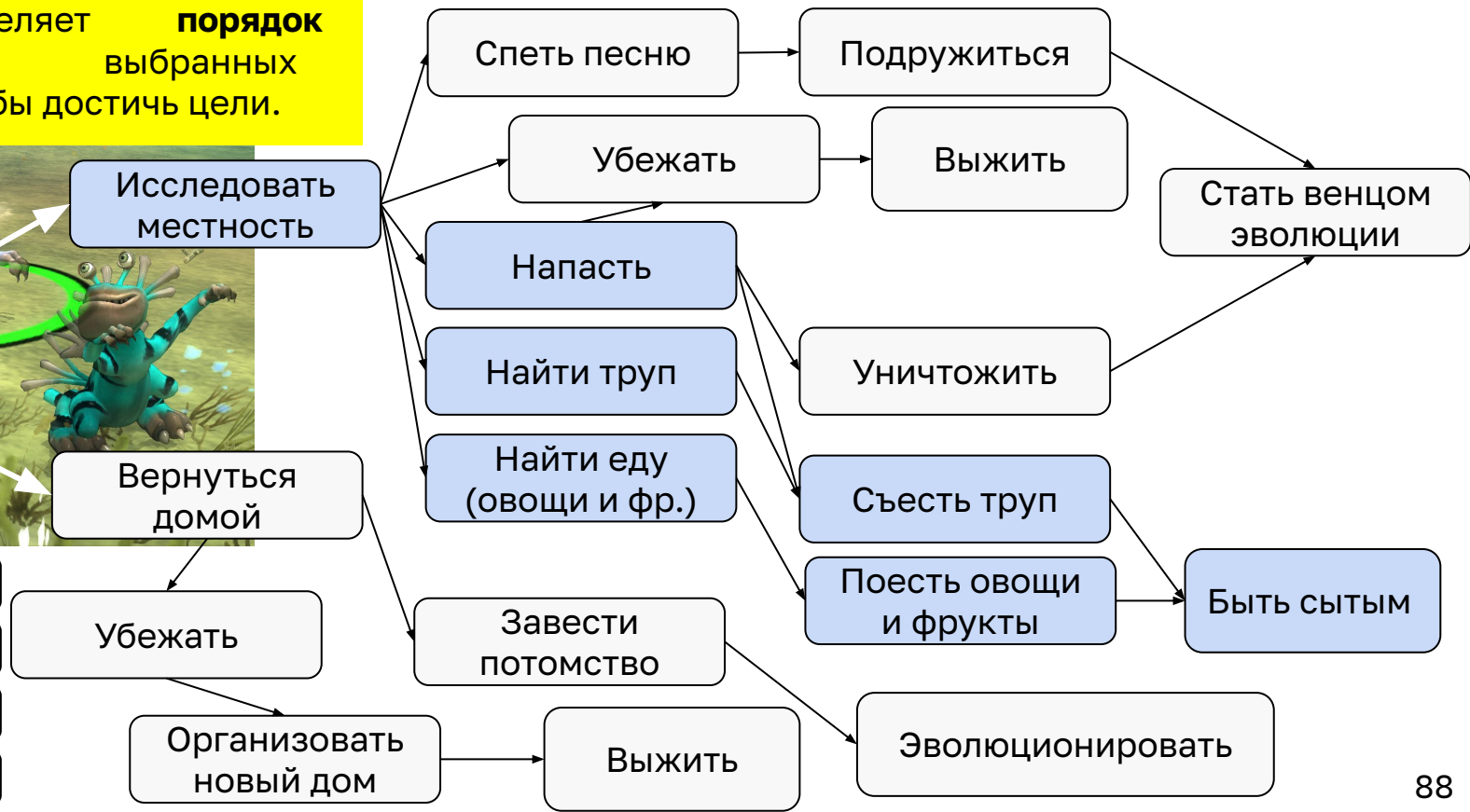
Планирование
последовательности действий:
ИИ определяет **порядок**
выполнения выбранных
действий, чтобы достичь цели.

Планирование

ИТМО



- Голод
- Выживание
- Эволюция
- Высшая цель



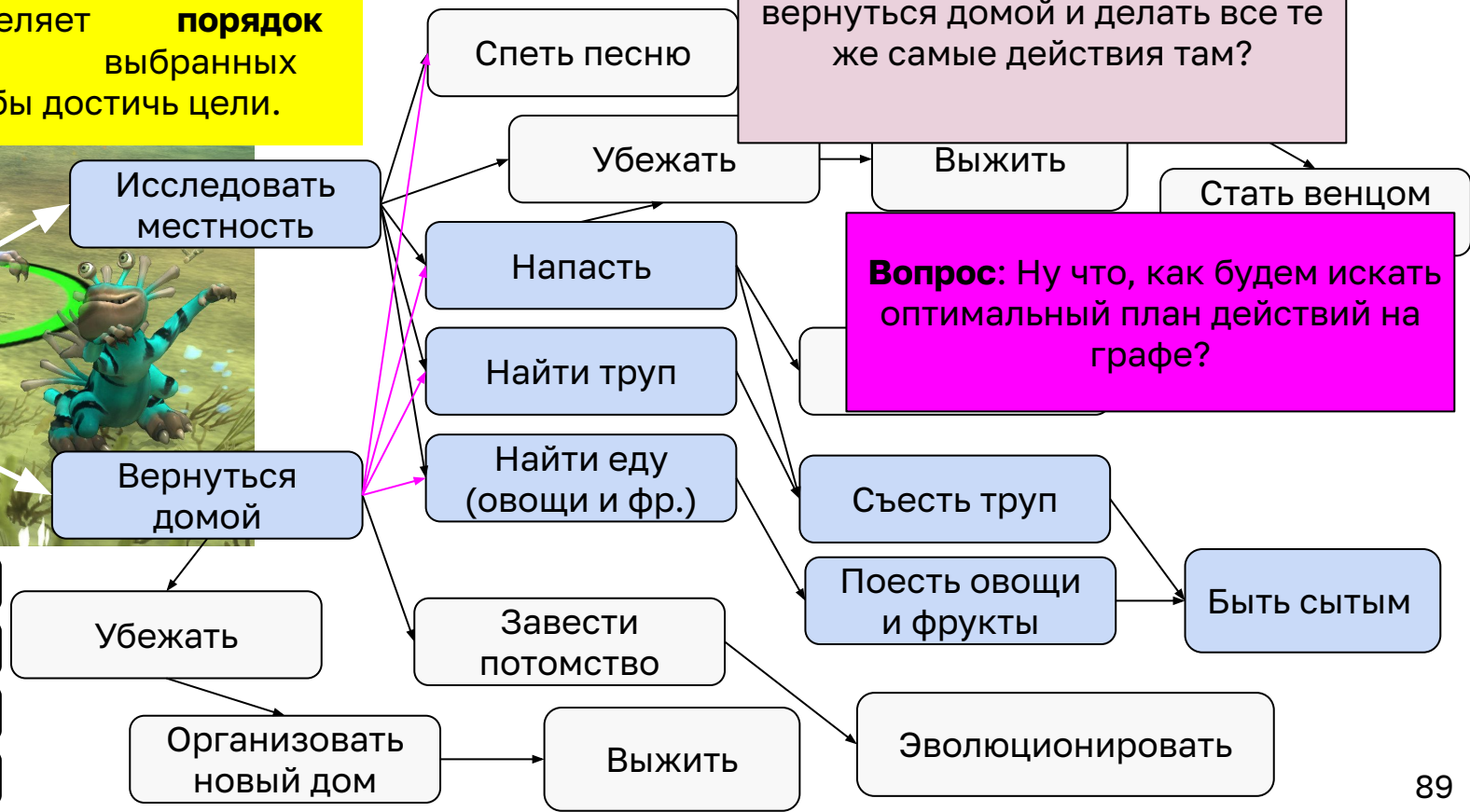
Этап 3.

Планирование
последовательности действий:
ИИ определяет **порядок**
выполнения выбранных
действий, чтобы достичь цели.



- Голод
- Выживание
- Эволюция
- Высшая цель

Планирование



Этап 3.

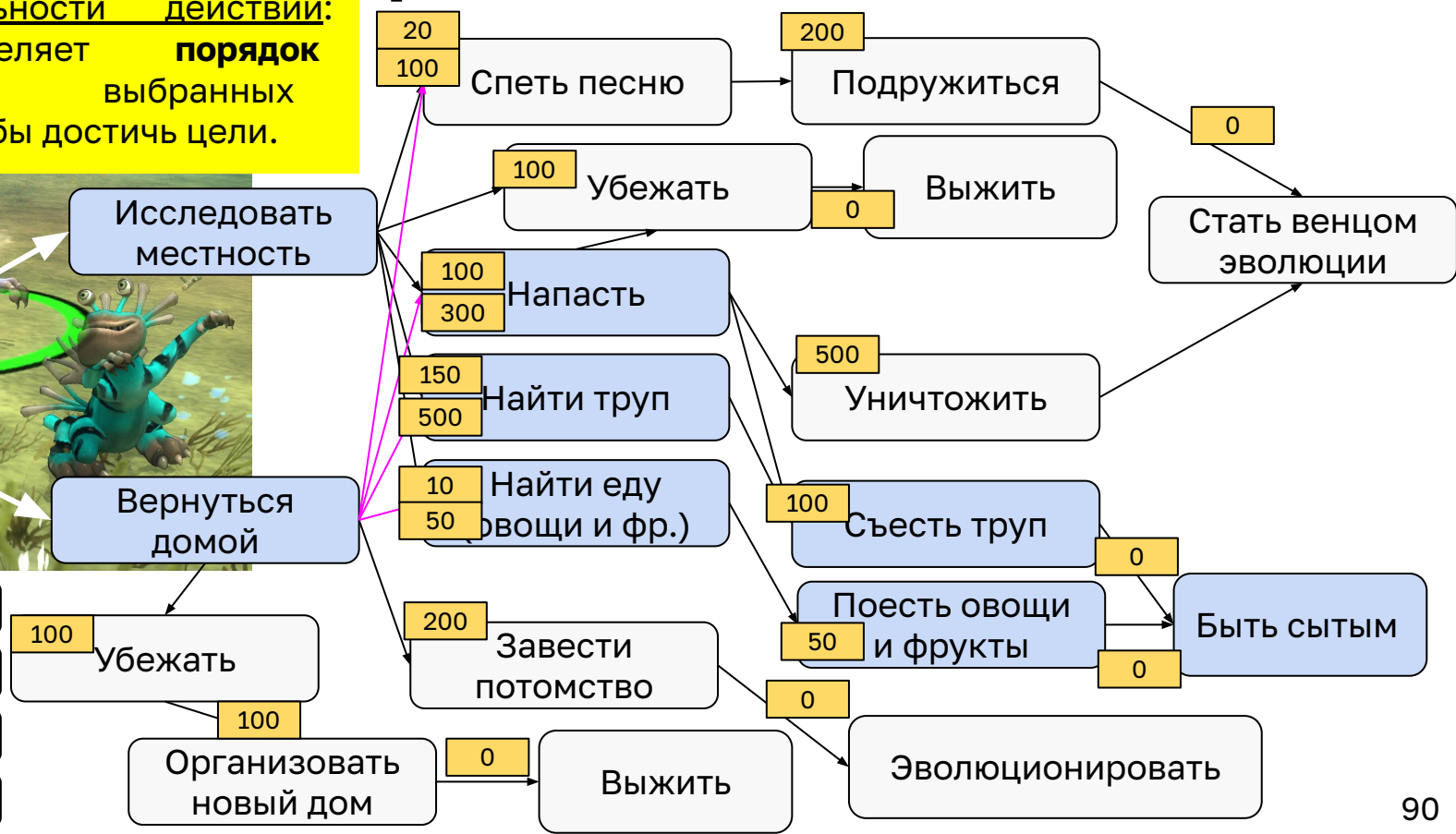
Планирование
последовательности действий:
ИИ определяет **порядок**
выполнения выбранных
действий, чтобы достичь цели.

Планирование

ИТМО



- Голод
- Выживание
- Эволюция
- Высшая цель



Этап 4.

Выполнение плана: ИИ следует определенному плану действий для достижения цели.

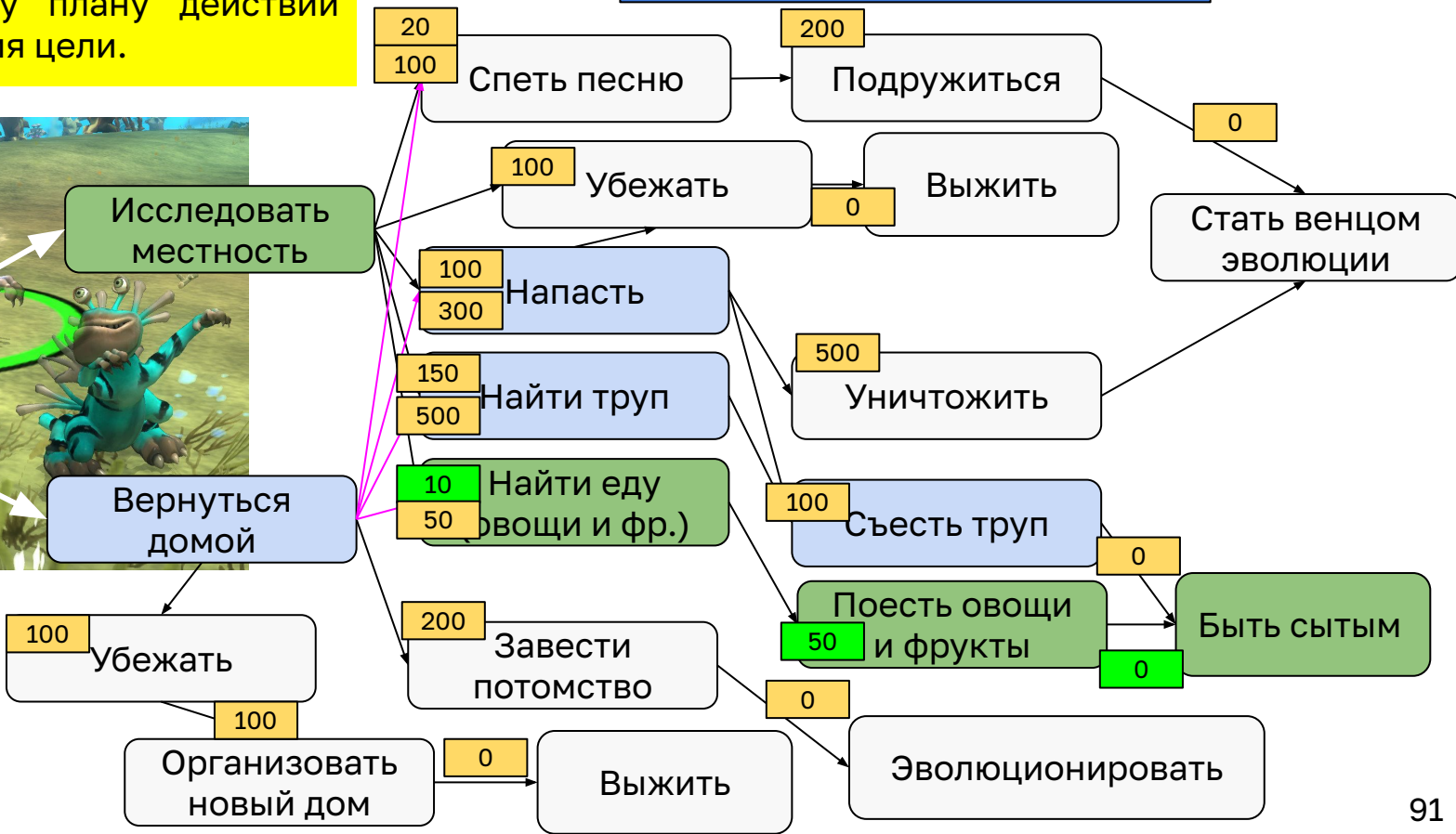
ировани

Используя A* ...

ГМО



- Голод**
- Выживание**
- Эволюция**
- Высшая цель**



Этап 4.

Выполнение плана: ИИ следует определенному плану действий для достижения цели.

Провани

Используя A* ...

ГМО

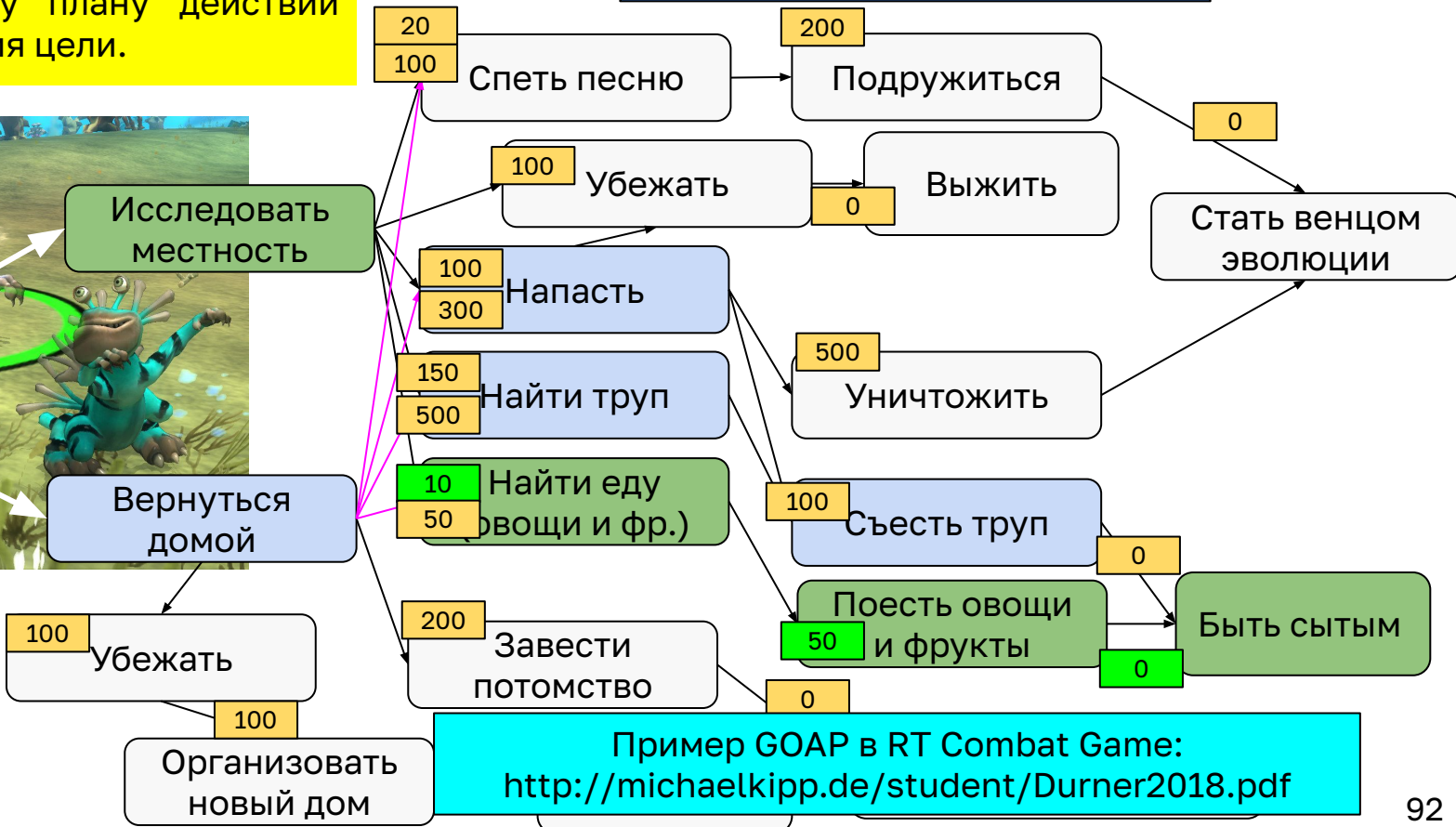


Голод

Выживание

Эволюция

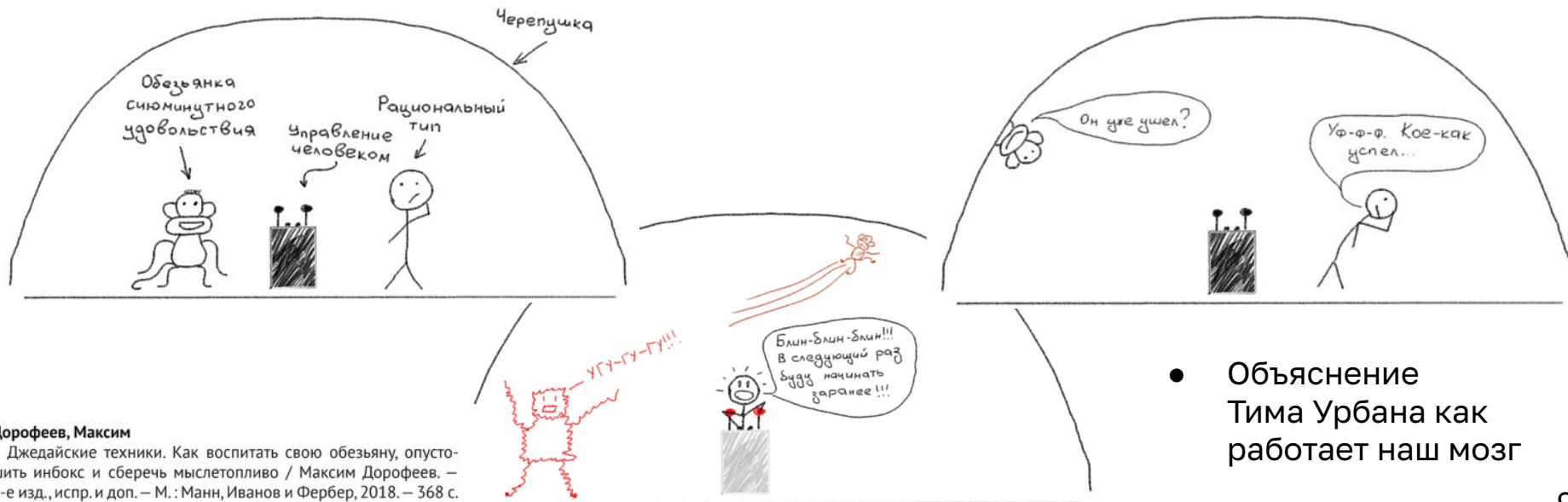
Высшая цель



Пример GOAP в RT Combat Game:
<http://michaelkipp.de/student/Durner2018.pdf>

ГЛАВА 3 _ Планирование

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



- Объяснение Тима Урбана как работает наш мозг

Дорофеев, Максим

Джедайские техники. Как воспитать свою обезьяну, опустошить инбокс и сберечь мыслотопливо / Максим Дорофеев. — 3-е изд., испр. и доп. — М.: Манн, Иванов и Фербер, 2018. — 368 с.

ISBN 978-5-00117-065-5

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.

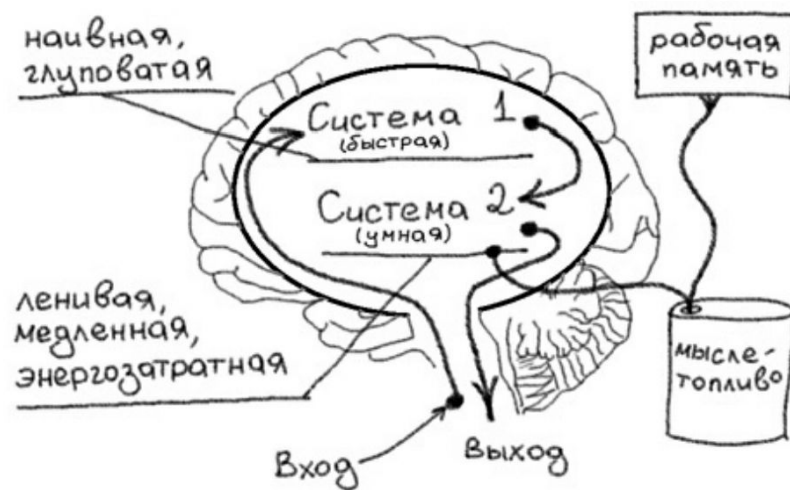
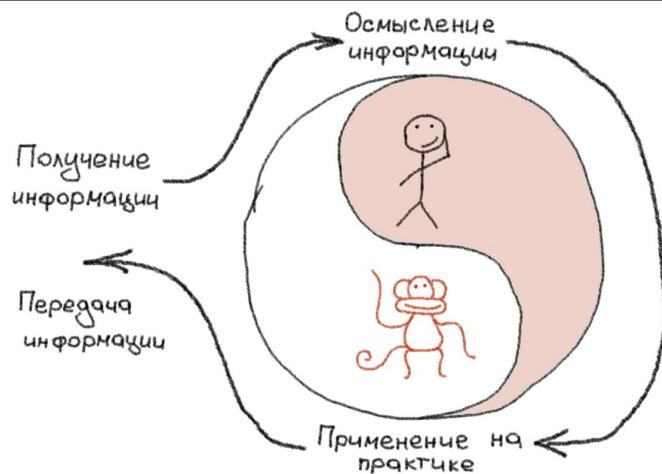


Рис. 4. Модель мышления по Даниэлю Канеману

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.

Пример, чтобы прояснить концепцию HTN, можно рассмотреть задачу приготовления обеда. Задача **"приготовить обед"** может быть разложена на такие подзадачи, как **"найти рецепт"**, **"собрать продукты"**, **"приготовить ингредиенты"**, **"готовить блюда"** и т.д. Каждая из этих подзадач может быть дальше разложена на еще более простые подзадачи. Например, задача **"приготовить ингредиенты"** может быть разделена на **"нарезать овощи"**, **"помыть фрукты"** и т.д. Таким образом, HTN позволяет системе последовательно разбивать сложные задачи на более простые шаги, что облегчает выполнение задачи.

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.

Различные
уровни
интеллекта

HTN позволяет создавать персонажей с разными уровнями интеллекта

Адаптивное
поведение

HTN может быть использован для создания адаптивного поведения в игре.

Обучение и
адаптация

HTN может быть использован в сочетании с методами машинного обучения для улучшения интеллекта персонажей. Например, можно использовать алгоритмы обучения с подкреплением для настройки или генерации HTN-планов

Искусственные
помощники

HTN также может использоваться для создания искусственных помощников в играх.

ГЛАВА 3 _ Планирование

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



Выполнить убийство

Найти цель

Уязвимости

Оружие

Подготовка

Убийство

Изучить окружение

Исследовать
ближайшую
локацию

Собрать
информацию о
врагах

Найти скрытые
проходы

ГЛАВА 3 _ Планирование

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



Выполнить убийство

Найти цель

Уязвимости

Оружие

Подготовка

Убийство

Изучить окружение

Исследовать
ближайшую
локацию

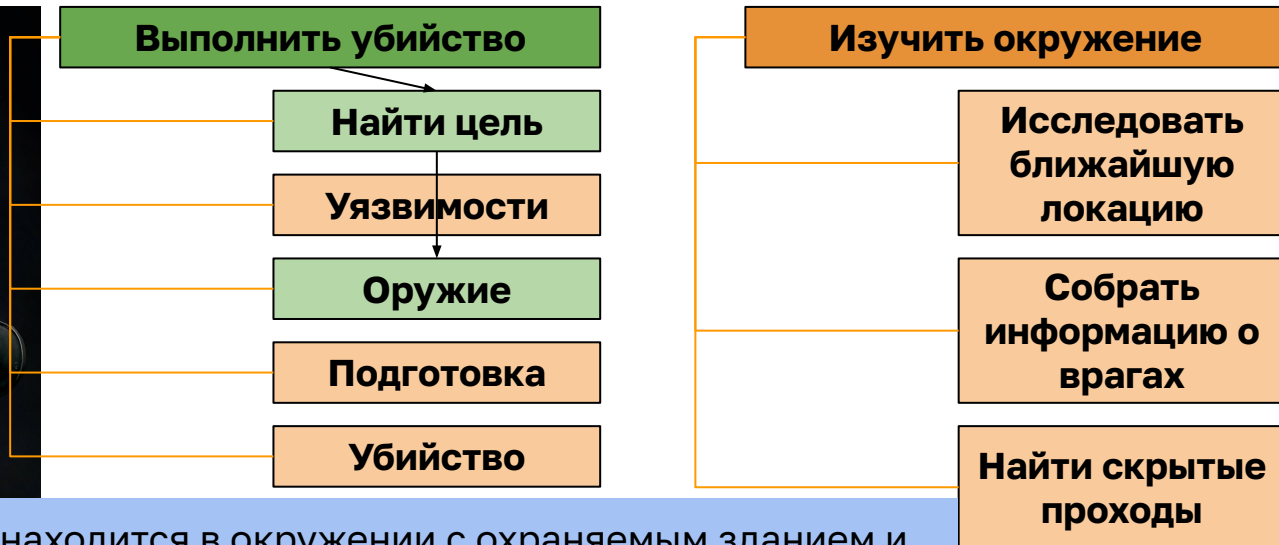
Собрать
информацию о
врагах

Найти скрытые
проходы

Предположим, что персонаж находится в окружении с охраняемым зданием и его текущая цель - выполнить убийство определенного целевого персонажа.

ГЛАВА 3 _ Планирование

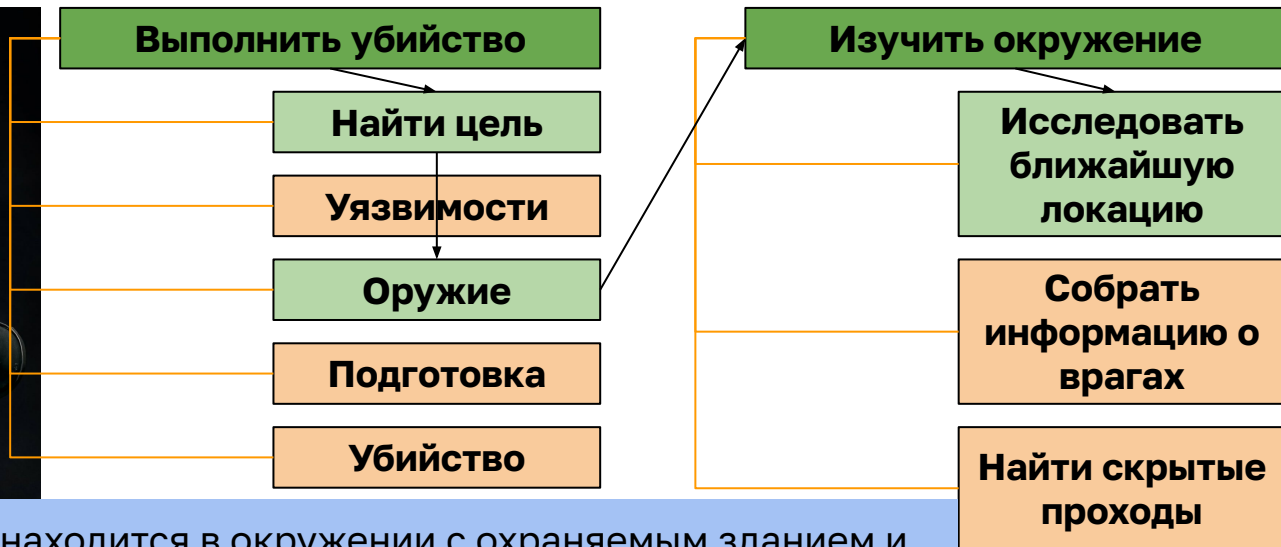
Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



Предположим, что персонаж находится в окружении с охраняемым зданием и его текущая цель - выполнить убийство определенного целевого персонажа.

ГЛАВА 3 _ Планирование

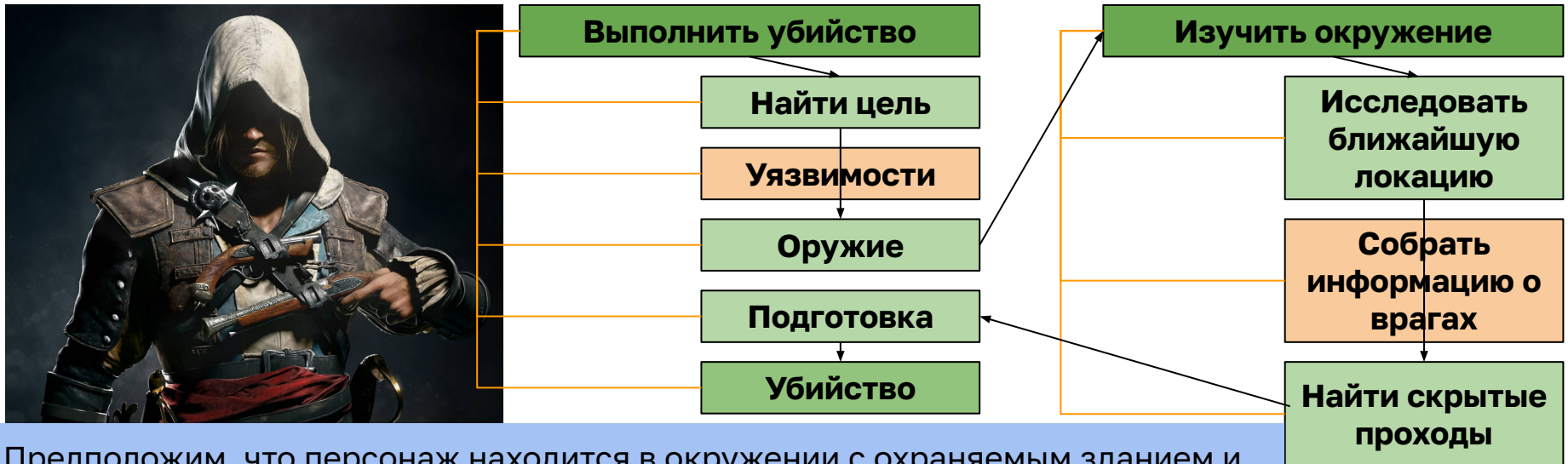
Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



Предположим, что персонаж находится в окружении с охраняемым зданием и его текущая цель - выполнить убийство определенного целевого персонажа.

ГЛАВА 3 _ Планирование

Hierarchical Task Network (HTN) planning - это метод планирования, используемый в искусственном интеллекте и робототехнике для решения сложных задач. Он основан на идее разбиения задач на более мелкие подзадачи и последующем их выполнении в иерархической структуре.



Предположим, что персонаж находится в окружении с охраняемым зданием и его текущая цель - выполнить убийство определенного целевого персонажа.

Оба подхода - **HNT** и **GOAP** - являются гибкими инструментами для **планирования поведения** и принятия решений в игре.

HNT является методом иерархического планирования, в котором задачи разбиваются на **подзадачи с определенными зависимостями и иерархической структурой**. Это позволяет персонажу структурировать и контролировать выполнение сложных задач путем декомпозиции и последовательного выполнения подзадач. HNT может быть рассмотрен как метод объединения и составления плана, который включает в себя различные уровни задач и их связи.

GOAP - это конкретная **реализация планирования**, которая сосредоточена на достижении конечной цели, определяя действия и операции, необходимые для ее достижения. GOAP стремится **найти план**, который приведет персонажа от текущего состояния к желаемой конечной цели, используя оценку этих действий и операций на основе наблюдаемого состояния системы. GOAP уделяет внимание выбору и комбинированию вариантов действий на основе конкретной проблемы и ограничений.

В конкретных реализациях планирования в игровом ИИ, элементы HNT и GOAP могут сосуществовать, и разработчики часто комбинируют их, чтобы достичь необходимого уровня гибкости, или варьируют подходы в зависимости от задачи и требований игры.

**Спасибо
за внимание!**

ITMO *re than a*
UNIVERSITY

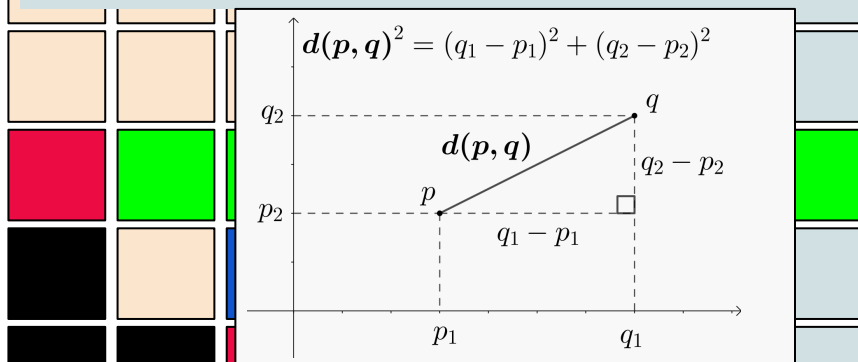
tatiana.atyapsheva@mail.ru
abrosimov.kirill.1999@mail.ru

Алгоритм A*.

1. Временные метки M, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

Модификация:

Добавляется эвристическая функция
Например, **Евклидово расстояние**



2) Новая BM = мин (прошлая BM, значение в рассматриваемой вершине + значение ребра + значение ЭФ)

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

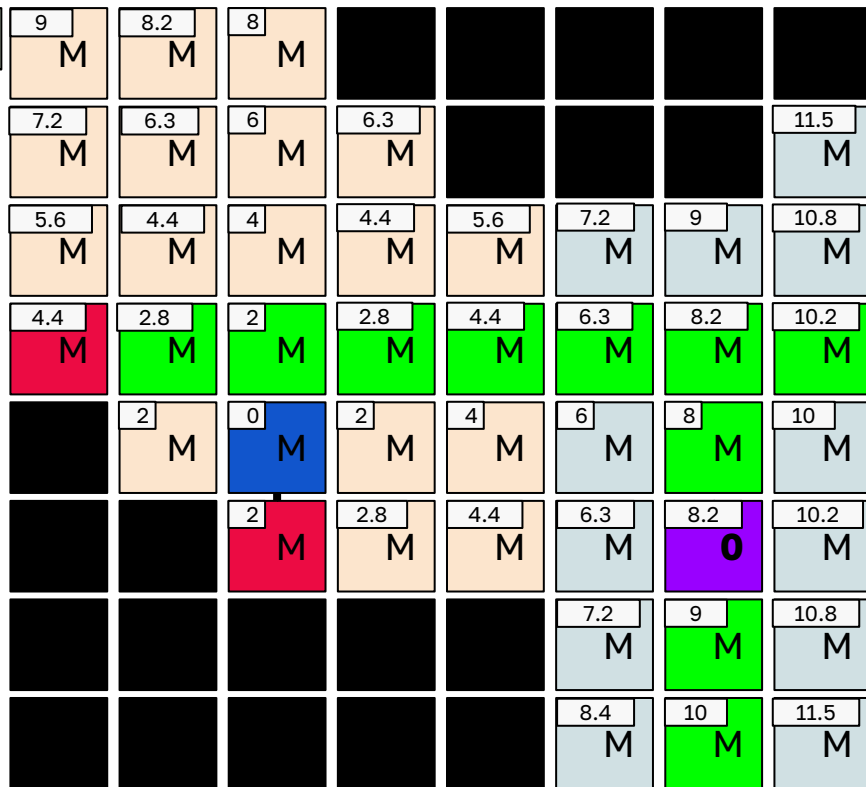
Напасть на врага – 10 ОД;

Посетить – 1 ОД

ГЛАВА 3 _ Навигация

Алгоритм A*.

1. Временные метки M, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная BM)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

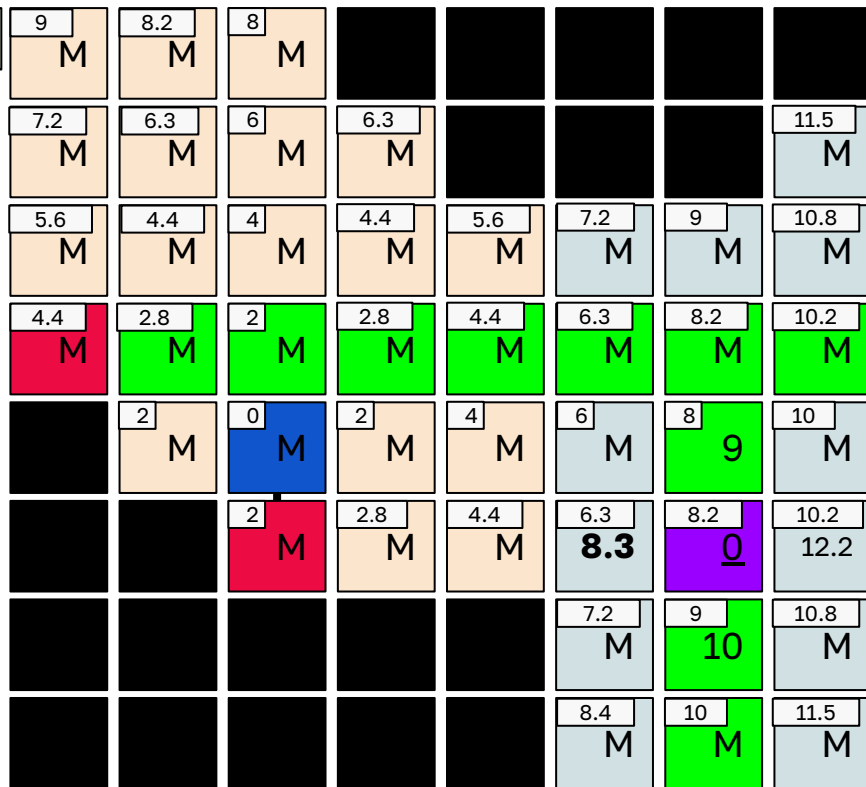
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

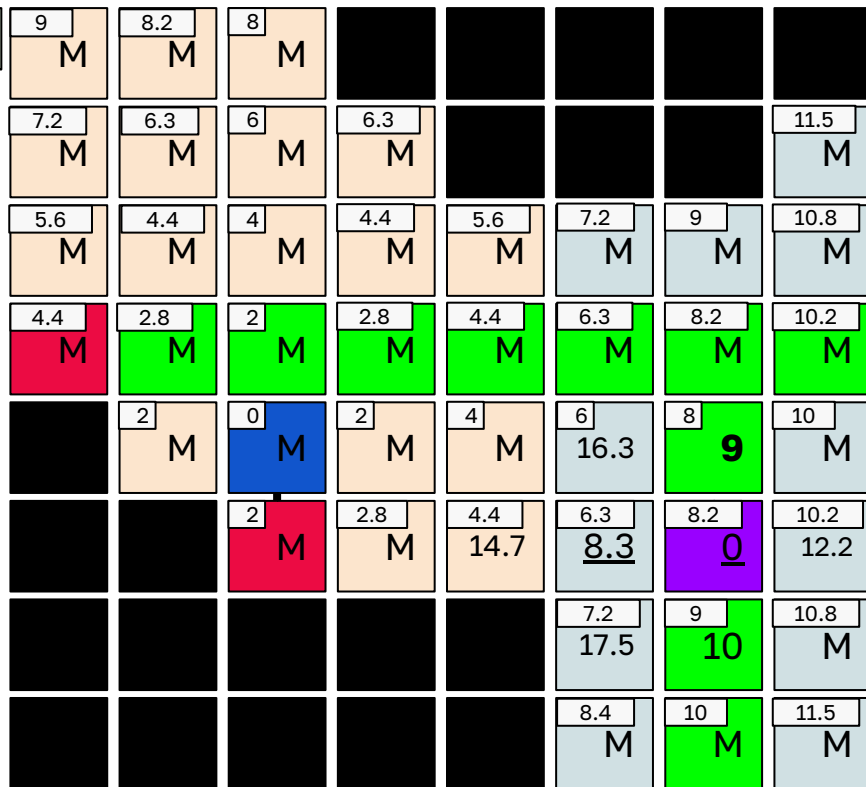
Посетить – 1 ОД

ЭФ: 2 * Евклидово расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

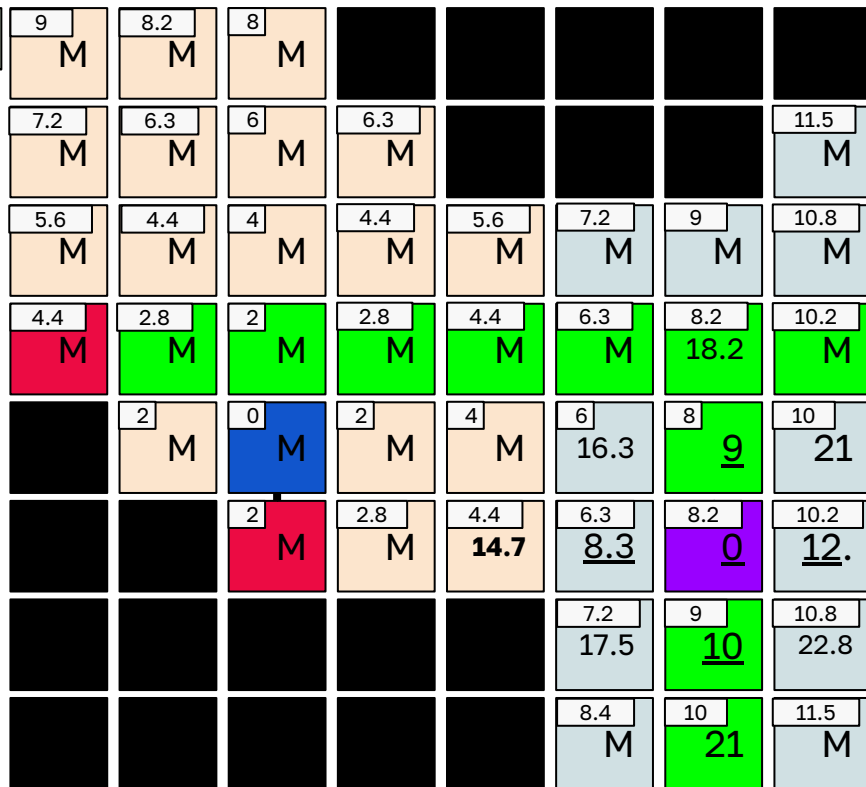
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

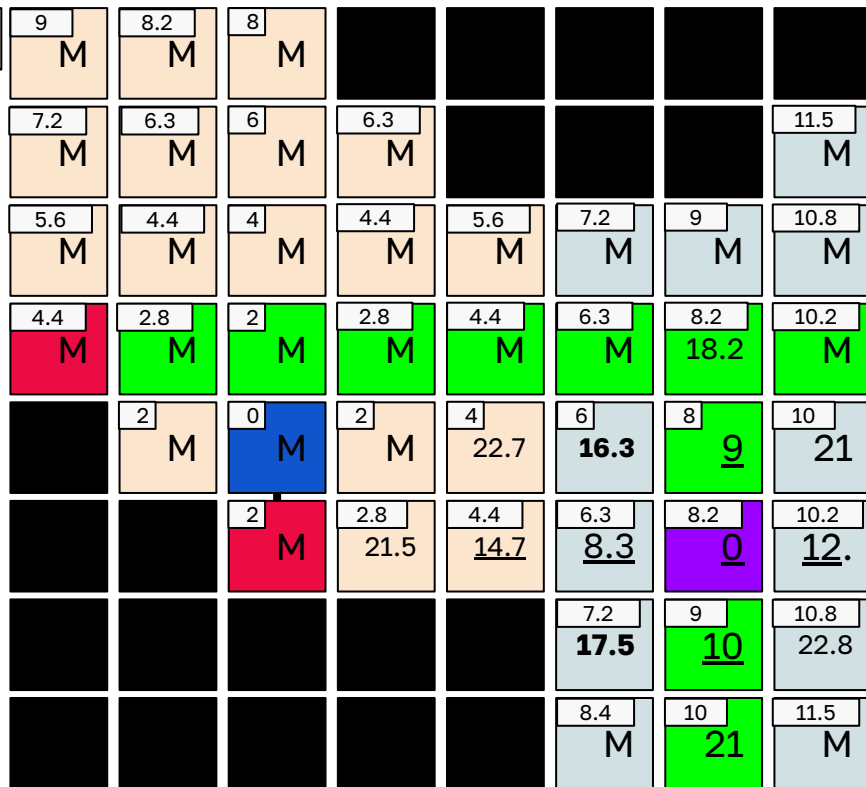
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

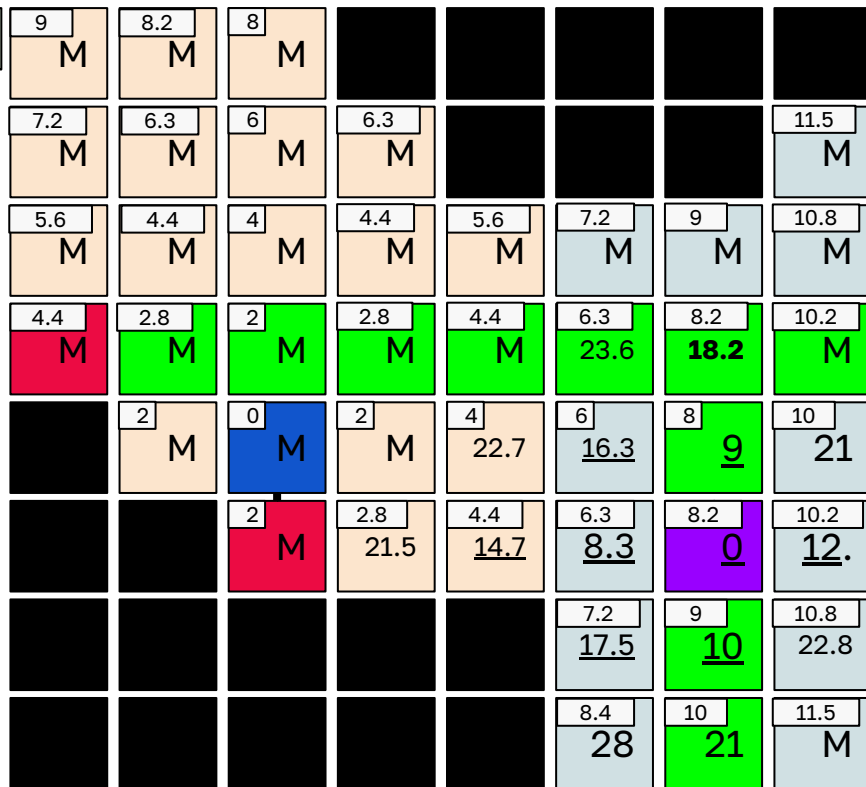
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

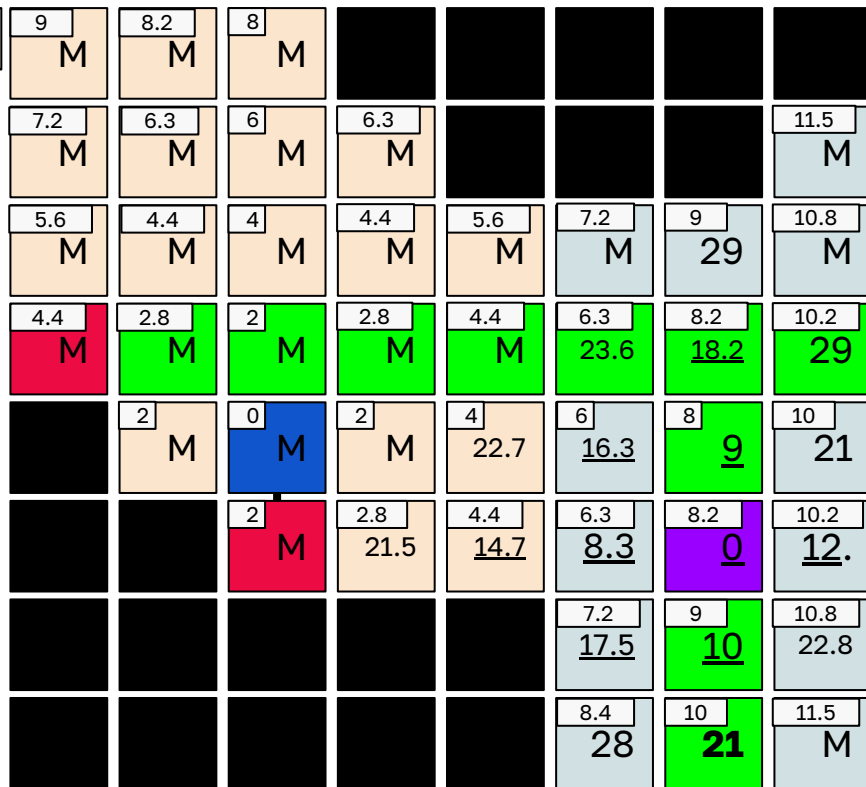
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

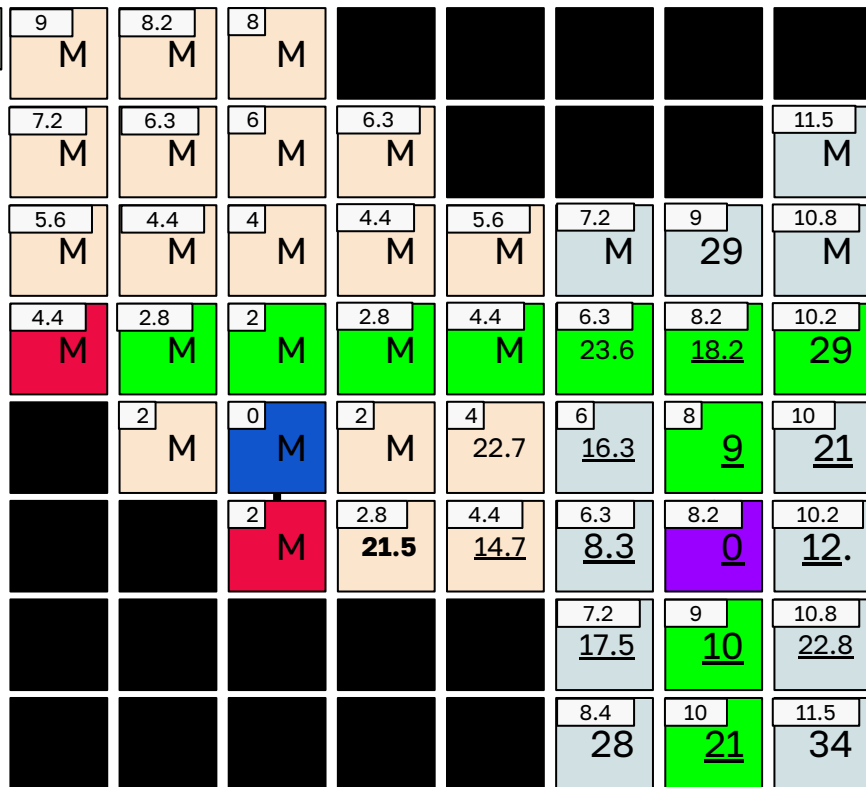
Посетить – 1 ОД

ЭФ: 2 * Евклидово расстояние до **конечной точки**

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

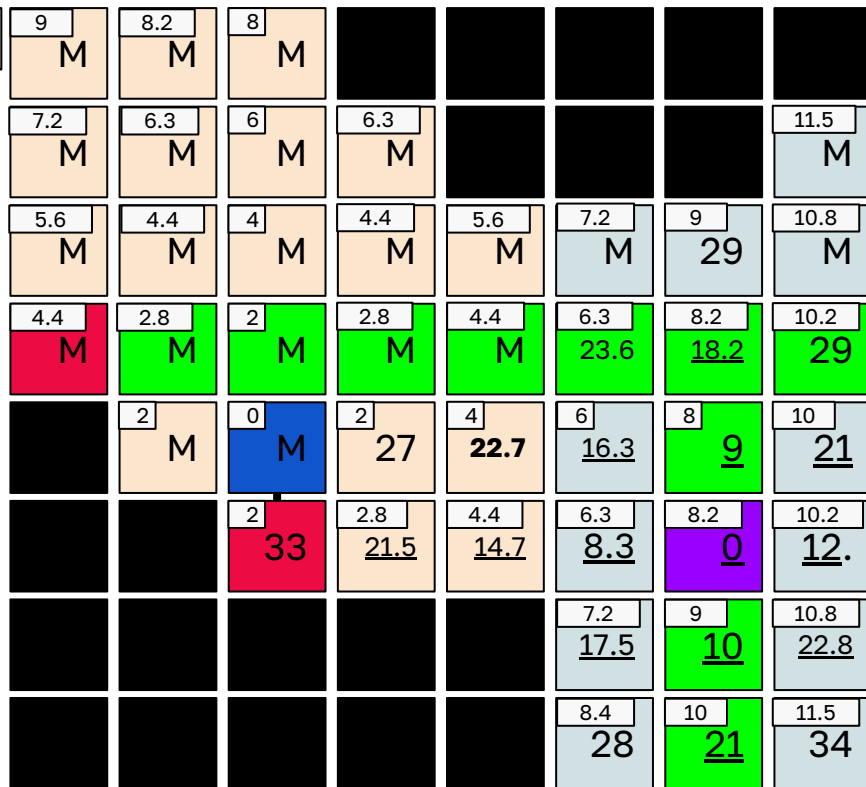
Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 М	5.6 М	7.2 М	9	10.8 М
4.4 М	2.8 М	2 М	2.8 М	4.4 М	6.3 М	8.2 М	10.2 М
	2 М	0 М	2 М	4 М	6 М	8 М	10 М
		2 М	2.8 М	4.4 М	6.3 М	8.2 М	10.2 М
					7.2 М	9 М	10.8 М
					8.4 М	10 М	11.5 М

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

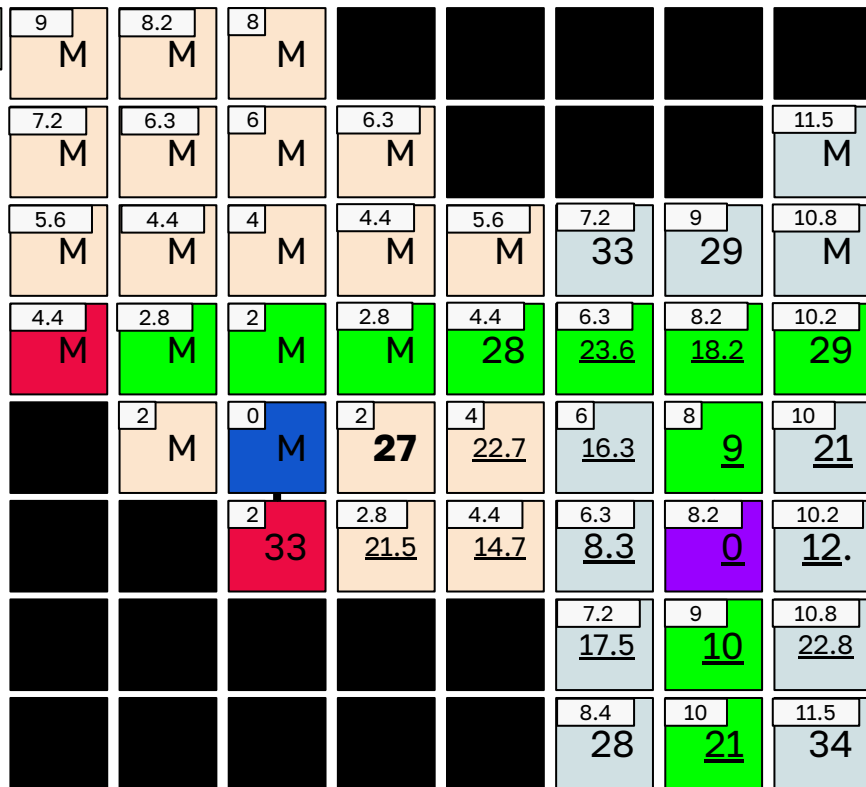
Посетить – 1 ОД

ЭФ: 2 * Евклидово расстояние до конечной точки

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)



Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 М	5.6 М	7.2 33	9 29	10.8 М
4.4 М	2.8 М	2 М	2.8 31	4.4 28	6.3 23.6	8.2 18.2	10.2 29
	2 М	0 М	2 27	4 22.7	6 16.3	8 9	10 21
		2 33	2.8 21.5	4.4 14.7	6.3 8.3	8.2 0	10.2 12.
					7.2 17.5	9 10	10.8 22.8
					8.4 28	10 21	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 М	5.6 38	7.2 33	9 29	10.8 М
4.4 М	2.8 М	2 М	2.8 31	4.4 28	6.3 23.6	8.2 18.2	10.2 29
	2 М	0 М	2 27	4 22.7	6 16.3	8 9	10 21
		2 33	2.8 21.5	4.4 14.7	6.3 8.3	8.2 0	10.2 12.
					7.2 17.5	9 10	10.8 22.8
					8.4 28	10 21	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 М	5.6 38	7.2 33	9 <u>29</u>	10.8 42
4.4 М	2.8 М	2 М	2.8 31	4.4 <u>28</u>	6.3 <u>23.6</u>	8.2 <u>18.2</u>	10.2 <u>29</u>
	2 М	0 М	2 <u>27</u>	4 <u>22.7</u>	6 <u>16.3</u>	8 <u>9</u>	10 <u>21</u>
		2 33	2.8 <u>21.5</u>	4.4 <u>14.7</u>	6.3 <u>8.3</u>	8.2 <u>0</u>	10.2 <u>12.</u>
					7.2 <u>17.5</u>	9 <u>10</u>	10.8 <u>22.8</u>
					8.4 <u>28</u>	10 <u>21</u>	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 39	5.6 38	7.2 33	9 29	10.8 42
4.4 М	2.8 М	2 34	2.8 31	4.4 28	6.3 23.6	8.2 18.2	10.2 29
	2 М	0 М	2 27	4 22.7	6 16.3	8 9	10 21
		2 33	2.8 21.5	4.4 14.7	6.3 8.3	8.2 0	10.2 12.
					7.2 17.5	9 10	10.8 22.8
					8.4 28	10 21	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 39	5.6 38	7.2 33	9 29	10.8 42
4.4 М	2.8 М	2 34	2.8 31	4.4 28	6.3 23.6	8.2 18.2	10.2 29
	2 М	0 34	2 27	4 22.7	6 16.3	8 9	10 21
		2 33	2.8 21.5	4.4 14.7	6.3 8.3	8.2 0	10.2 12.
					7.2 17.5	9 10	10.8 22.8
					8.4 28	10 21	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 39	5.6 38	7.2 33	9 29	10.8 42
4.4 М	2.8 М	2 34	2.8 31	4.4 28	6.3 23.6	8.2 18.2	10.2 29
	2 М	0 34	2 27	4 22.7	6 16.3	8 9	10 21
		2 33	2.8 21.5	4.4 14.7	6.3 8.3	8.2 0	10.2 12.
					7.2 17.5	9 10	10.8 22.8
					8.4 28	10 21	11.5 34

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$

ГЛАВА 3 _ Навигация

Алгоритм А*.

1. Временные метки М, отсчет 0.
Определяем ЭФ.
2. Обновляем доступные вершины (временные метки)
3. Фиксируем обработанную вершину (временная метка → постоянная)
4. Выбираем следующую вершину для обработки (минимальная ВМ)
5. Повторяем шаги 2-4, пока на целевой вершине не будет постоянной метки (ПМ)

9 М	8.2 М	8 М					
7.2 М	6.3 М	6 М	6.3 М				11.5 М
5.6 М	4.4 М	4 М	4.4 39	5.6 38	7.2 <u>33</u>	9 <u>29</u>	10.8 42
4.4 М	2.8 М	2 34	2.8 <u>31</u>	4.4 <u>28</u>	6.3 <u>23.6</u>	8.2 <u>18.2</u>	10.2 <u>29</u>
	2 М	0 <u>34</u>	2 <u>27</u>	4 <u>22.7</u>	6 <u>16.3</u>	8 <u>9</u>	10 <u>21</u>
		2 <u>33</u>	2.8 <u>21.5</u>	4.4 <u>14.7</u>	6.3 <u>8.3</u>	8.2 <u>0</u>	10.2 <u>12.</u>
					7.2 <u>17.5</u>	9 <u>10</u>	10.8 <u>22.8</u>
					8.4 <u>28</u>	10 <u>21</u>	11.5 <u>34</u>

Правила (стоимость):

Дорога – 1 ОД

“Своя” местность – 2 ОД

“Чужая” местность – 4 ОД

Напасть на врага – 10 ОД;

Посетить – 1 ОД

ЭФ: $2 * \text{Евклидово расстояние до конечной точки}$